Dossier Projet - RNCP 5

Titre professionnel : Developpeur Web et Web Mobile (DWWM)

Page de garde

- **Titre du projet** : FloraShop — Pivoine & Lilas, Application e-commerce pour artisan fleuriste
- **Nom et prenom du candidat** : Jaille Dimitri
- **Date** : Mars 2026
- **Titre professionnel vise** : Developpeur Web et Web Mobile (DWWM) - RNCP Niveau 5 (Code RNCP 37674)
- **Organisme de formation** : Holberton School
- **Session** : 2025-2026

Fig. 1 : Logo Holberton School

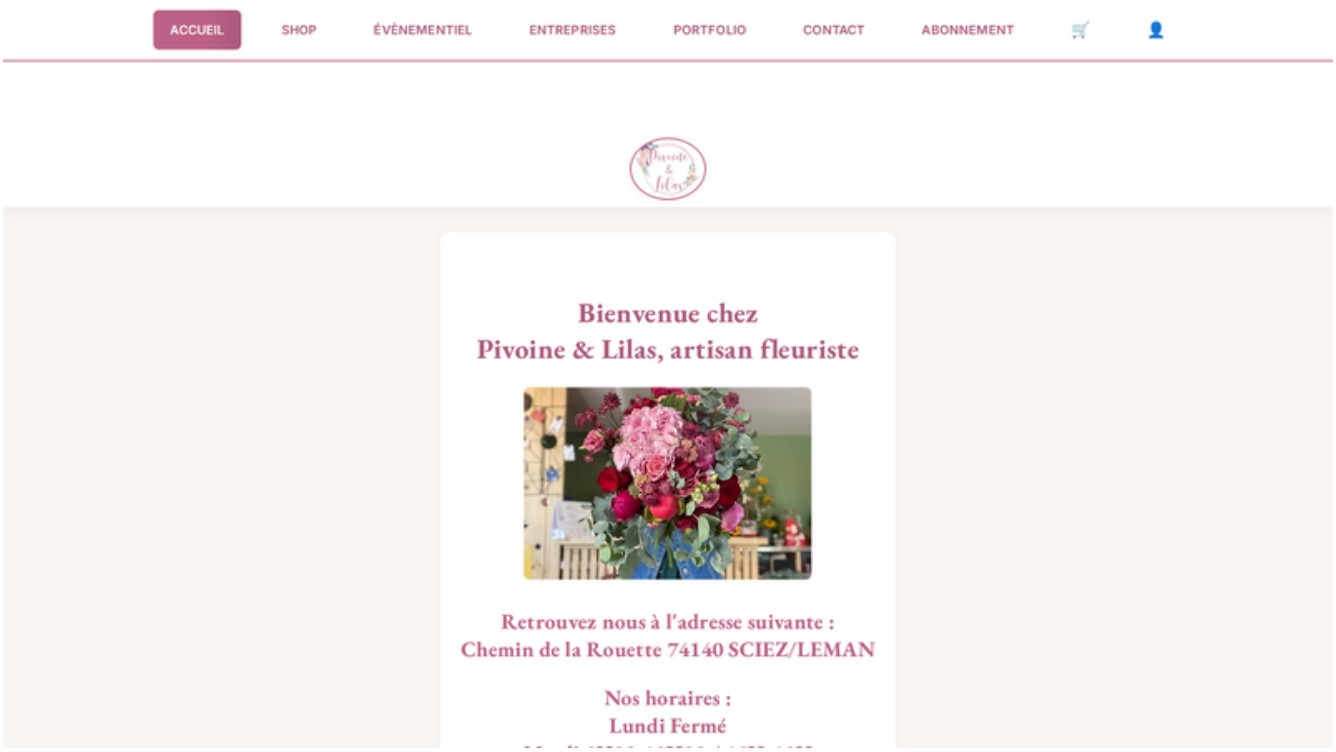


Fig. 2 : Page d accueil FloraShop - Pivoine & Lilas

Remerciements

Je tiens a remercier l ensemble de l equipe pedagogique de Holberton School pour leur accompagnement tout au long de cette formation. Leurs conseils techniques et methodologiques m ont permis de progresser et de mener ce projet a bien.

Je remercie tout particulierement la mere de Mattieu Mourroux, gerante de Pivoine & Lilas, pour sa confiance dans la realisation de cette application e-commerce. Ce projet est ne de la volonte d aider son entreprise a developper son activite d artisan fleuriste en lui offrant une presence en ligne professionnelle. Sa disponibilite lors de la definition du cahier des charges et ses retours sur les besoins fonctionnels ont ete essentiels a la reussite du projet.

Merci a Mattieu Mourroux, ami et co-developpeur sur ce projet, pour sa collaboration precieuse tout au long du developpement.

Merci a mes camarades de promotion pour les echanges constructifs, le partage de connaissances et l entraide qui ont contribue a enrichir mon apprentissage.

Enfin, je remercie mes proches pour leur soutien et leurs encouragements constants tout au long de cette formation.

Resume du projet

FloraShop est une application web e-commerce full-stack developpee pour Pivoine & Lilas, artisan fleuriste. Le projet consiste en la conception et la realisation d une boutique en ligne permettant la vente de fleurs fraiches, compositions florales, vases, parfums, plantes d interieur, accessoires et coffrets cadeaux.

L application offre une experience utilisateur complete : catalogue de produits avec filtres par categories et recherche, panier d achat dynamique, paiement securise via Stripe, gestion de comptes utilisateurs avec authentification JWT, et un tableau de bord d administration pour la gestion des produits, categories et utilisateurs.

Cote front-end, l interface est responsive et construite avec HTML5, CSS3 (design vintage aux couleurs de la marque #bc6288) et JavaScript vanilla. Cote back-end, l application repose sur Python Flask avec une architecture REST API documentee via Swagger (Flask-RESTX), une base de donnees PostgreSQL geree par SQLAlchemy ORM avec migrations Alembic, et une integration de paiement Stripe complete (Payment Intents + webhooks).

Le projet couvre l ensemble des competences du referentiel DWWM : maquettage d interfaces, realisation d interfaces statiques et dynamiques responsives, mise en place d une base de donnees relationnelle, developpement de composants d acces aux donnees et de composants metier cote serveur, le tout avec une approche securisee (hashage de mots de passe, JWT, validation des entrees, verification de signatures webhooks).

Table des matieres

1. Liste des competences du referentiel couvertes par le projet
2. Contexte du projet
   - 2.1 Type de cahier des charges
   - 2.2 Presentation de l entreprise et du service
   - 2.3 Cahier des charges / Expression des besoins
   - 2.4 Contraintes du projet et livrables attendus
   - 2.5 Environnement humain, technique et objectifs de qualite
3. Elements significatifs cote front-end
4. Elements significatifs cote back-end
5. Conclusion
6. Bibliographie
7. Annexes
   - 7.1 Swagger UI (documentation API)
   - 7.2 Donnees initiales
   - 7.3 Historique des migrations
   - 7.4 Glossaire technique
   - 7.5 Structure du projet

Liste des competences du referentiel couvertes par le projet

AT1 - Developper la partie front-end d une application web ou web mobile securisee

- ☒ CP1 - Maquetter des interfaces utilisateur web ou web mobile
  - Maquettes des pages : accueil, boutique, panier, checkout, administration, compte, inscription, contact, portfolio, evenementiel, entreprises, abonnement
  - Design responsive (desktop / tablette / mobile)
- ☒ CP2 - Realiser des interfaces utilisateur statiques web ou web mobile
  - 16 templates HTML5/CSS3 (Jinja2), 4 fichiers CSS (932 lignes), charte graphique vintage coherente
  - Navigation responsive avec barre fixe, logo, menu, icones panier et profil
- ☒ CP3 - Developper la partie dynamique des interfaces utilisateur web ou web mobile
  - JavaScript vanilla (api.js : 261 lignes, auth.js : 59 lignes)
  - Gestion dynamique du panier (ajout, suppression, calcul total), filtres produits, recherche, authentification cote client, integration Stripe.js

AT2 - Developper la partie back-end d une application web ou web mobile securisee

- ☒ CP4 - Mettre en place une base de donnees relationnelle
  - PostgreSQL avec 7 tables : users, categories, products, orders, order_items, reviews, prices
  - Relations : Category→Products (1-N), Order→OrderItems (1-N), User→Orders (1-N), User→Reviews (1-N)
  - Gestion des migrations avec Flask-Migrate (Alembic) : 6 versions de migration
- ☒ CP5 - Developper des composants d acces aux donnees SQL et NoSQL
  - Pattern Repository : CategoryRepository, ProductRepository, PriceRepository, UserRepository, ReviewRepository
  - ORM SQLAlchemy avec operations CRUD completes, validation d ID, verification d unicite
  - Facade de services agregant tous les repositories
- ☒ CP6 - Developper des composants metier cote serveur
  - REST API documentee via Flask-RESTX (Swagger UI auto-generee)
  - StripeService : creation de PaymentIntent, confirmation de paiement, gestion webhooks
  - Authentification JWT (PyJWT), gestion des roles (admin/utilisateur)
  - Regles metier : calcul de totaux, gestion des statuts de commande (pending/paid/failed/cance

---

Contexte du projet

Type de cahier des charges

☒ Projet en entreprise
☐ Projet en formation

Presentation de l entreprise et du service

Le projet FloraShop a ete realise dans le cadre de la formation Developpeur Web et Web Mobile (DWWM) a Holberton School pour le compte de l entreprise **Pivoine & Lilas**, artisan fleuriste geree par la mere de Mattieu Mourroux. Ce projet est ne d une demarche solidaire : aider la mere d un ami et co-developpeur a developper son activite en lui concevant une boutique en ligne professionnelle. Il s agit de la conception et du developpement d une application e-commerce complete permettant a l entreprise de vendre ses produits en ligne.

L activite de Pivoine & Lilas comprend : - La vente de fleurs fraiches et compositions florales - La vente de vases, parfums, plantes d interieur, accessoires et coffrets cadeaux - Un service d abonnement floral (livraisons regulieres) - Des prestations evenementielles (mariages, evenements d entreprise) - Des services B2B pour les entreprises

Cahier des charges / Expression des besoins

Objectif principal : Developper une application web e-commerce full-stack permettant a un artisan fleuriste de vendre ses produits en ligne.

Besoins fonctionnels :

1. **Catalogue produits** : Affichage des produits organises par categories (Fleurs Fraiches, Vases, Parfums, Plantes d Interieur, Accessoires, Compositions, Cadeaux), avec filtres, recherche par nom, et tri par prix
2. **Panier d achat** : Ajout/suppression de produits, modification des quantites, calcul automatique du total
3. **Paiement en ligne** : Integration Stripe pour le paiement securise par carte bancaire (EUR)
4. **Gestion des comptes** : Inscription, connexion, deconnexion, suppression de compte
5. **Tableau d administration** : Interface CRUD pour gerer les produits, categories et utilisateurs (accessible uniquement aux administrateurs)
6. **API REST** : Endpoints documentes via Swagger pour toutes les operations CRUD
7. **Pages vitrine** : Accueil, portfolio, evenementiel, entreprises, contact
8. **Abonnement floral** : Page de souscription a un abonnement de livraisons regulieres

Besoins non-fonctionnels :

- Interface responsive (desktop, tablette, mobile)
- Securite : hashage des mots de passe, authentification JWT, validation des entrees
- Performance : chargement rapide des pages, requetes optimisees
- Maintenabilite : architecture MVC, pattern repository, code documente

Contraintes du projet et livrables attendus

Contraintes :

- Delai : projet a realiser dans le temps imparti de la formation Holberton School
- Technologies retenues : Python/Flask pour le back-end, HTML/CSS/JS pour le front-end
- Base de donnees relationnelle obligatoire (PostgreSQL)
- Paiement en ligne fonctionnel (API Stripe en mode test)
- Application deployable en local

Livrables attendus :

- Code source complet (depot Git)
- Application fonctionnelle avec toutes les pages et fonctionnalites
- Base de donnees initialisee avec donnees de demonstration (7 categories, 10 produits, 1 admin)
- Documentation technique (Swagger UI, README)
- Presentation orale du projet

Environnement humain, technique et objectifs de qualite

Equipe / collaborateurs :

- Developpeur : Jaille Dimitri
- Developpeur : Mattieu Mourroux
- Commanditaire : Pivoine & Lilas (artisan fleuriste)
- Encadrement : Equipe pedagogique Holberton School

Environnement technique (stack, outils, methodes) :

Categorie	Technologies
Langages front-end	HTML5, CSS3, JavaScript (ES6+)
Frameworks front-end	Jinja2 (moteur de templates), Stripe.js v3
Langages back-end	Python 3.10
Frameworks back-end	Flask 3.1.0, Flask-RESTX 1.3.0, Flask-SQLAlchemy 3.1.1, Flask-Migrate 4.1.0
Base de donnees	PostgreSQL (via psycopg2), SQLAlchemy 2.0.23 (ORM)
Outils de versioning	Git, GitHub
Outils de deploiement	Flask development server, python-dotenv pour la configuration
Outils de test	coverage 7.6.12, scripts de test BDD (test_db.py, test_db.sh)
Autres	Stripe API (paiement), PyJWT (authentification), Werkzeug (hashage), Google reCAPTCHA, Flask-CORS, Swagger/OpenAPI 3.0

Objectifs de qualite :

- Code structure selon le pattern MVC (Models / Views-Templates / Controllers-Routes)
- Pattern Repository pour l acces aux donnees (separation des responsabilites)
- Facade de services pour centraliser la logique metier
- API RESTful documentee automatiquement via Swagger UI
- Gestion des erreurs avec try/catch et rollback des transactions
- Variables sensibles externalisees dans des fichiers .env

Methodologie de gestion de projet :

- Methode iterative : developpement incremental des fonctionnalites
- Versioning Git avec commits reguliers
- Gestion des migrations de base de donnees avec Alembic (Flask-Migrate)
- 6 migrations successives tracant l evolution du schema (ajout champ password etendu, ajout promotions, ajout is_on_sale, suppression product_id des reviews)

Elements significatifs cote front-end

Maquettes de l application (responsive)

L application comprend 16 pages HTML, chacune maquettee selon la charte graphique de Pivoine & Lilas :

Page	Fichier	Description
Accueil	accueil.html	Page d accueil avec presentation de la marque
Boutique	shop.html (461 lignes)	Catalogue produits avec filtres, recherche, panier
Panier	panier.html (375 lignes)	Gestion du panier avec quantites et total
Paiement	checkout.html (588 lignes)	Formulaire de paiement Stripe avec resume commande
Paiement alternatif	payment.html (406 lignes)	Page de paiement alternative
Abonnement	subscription.html (182 lignes)	Page d abonnement floral
Paiement abonnement	subscription_payment.html (593 lignes)	Paiement recurrent Stripe
Administration	admin.html (748 lignes)	Dashboard CRUD (categories, produits, utilisateurs)
Connexion	account.html (149 lignes)	Formulaire de connexion

Page	Fichier	Description
Inscription	register.html (127 lignes)	Formulaire d inscription avec reCAPTCHA
Contact	contact.html (114 lignes)	Page de contact
Entreprises	entreprises.html (114 lignes)	Services B2B
Evenementiel	evenementiel.html (112 lignes)	Services evenementiels
Portfolio	portfolio.html (138 lignes)	Galerie de realisations
Mot de passe oublie	forgot-password.html (83 lignes)	Recuperation de mot de passe
Verification code	verify-code.html (34 lignes)	Saisie du code de verification

Schema d enchainement des maquettes

```
Accueil ──> Boutique ──> Panier ──> Checkout ──> Paiement Stripe
   │            │                                      │
   │            └──> Detail produit                    └──> Confirmation
   │
   ├──> Evenementiel
   ├──> Entreprises
   ├──> Portfolio
   ├──> Contact
   ├──> Abonnement ──> Paiement abonnement
   │
   ├──> Connexion ──> Profil (deconnexion, suppression)
   ├──> Inscription (reCAPTCHA)
   ├──> Mot de passe oublie ──> Verification code
   │
   └──> Administration (si admin)
            ├──> CRUD Categories
            ├──> CRUD Produits
            └──> CRUD Utilisateurs
```

Navigation commune a toutes les pages via la barre de navigation fixe : Accueil | Shop | Evenementiel | Entreprises | Portfolio | Contact | Abonnement | Panier | Admin (si admin) | Profil

Captures d ecran de l interface utilisateur (responsive)

Les captures d ecran ci-dessous presentent les pages principales de l application en version desktop (1280x800) et mobile (393x852, simulation iPhone).

Page d accueil — Presentation de la marque Pivoine & Lilas avec navigation principale.

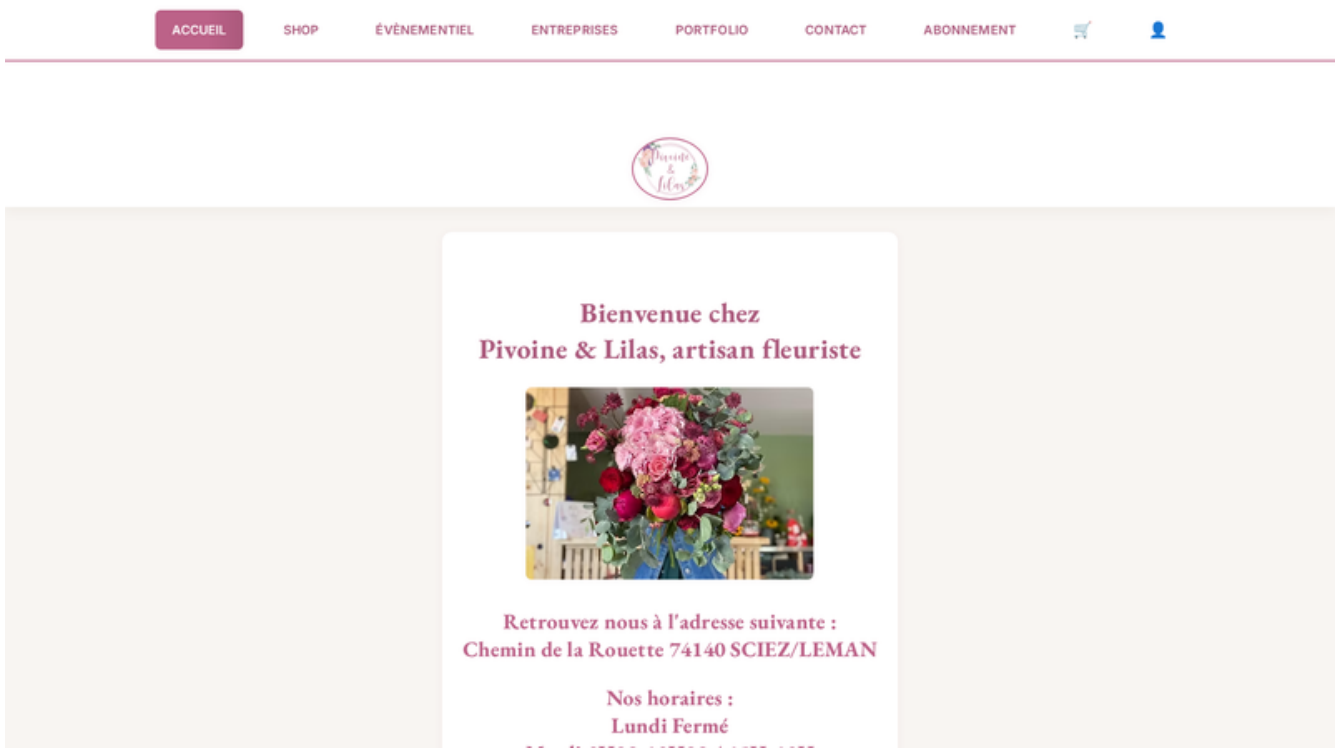


Fig. 3 : Page d accueil - Desktop

Fig. 4 : Page d'accueil - Mobile

Page boutique — Grille de produits avec filtres (categories, prix, recherche).

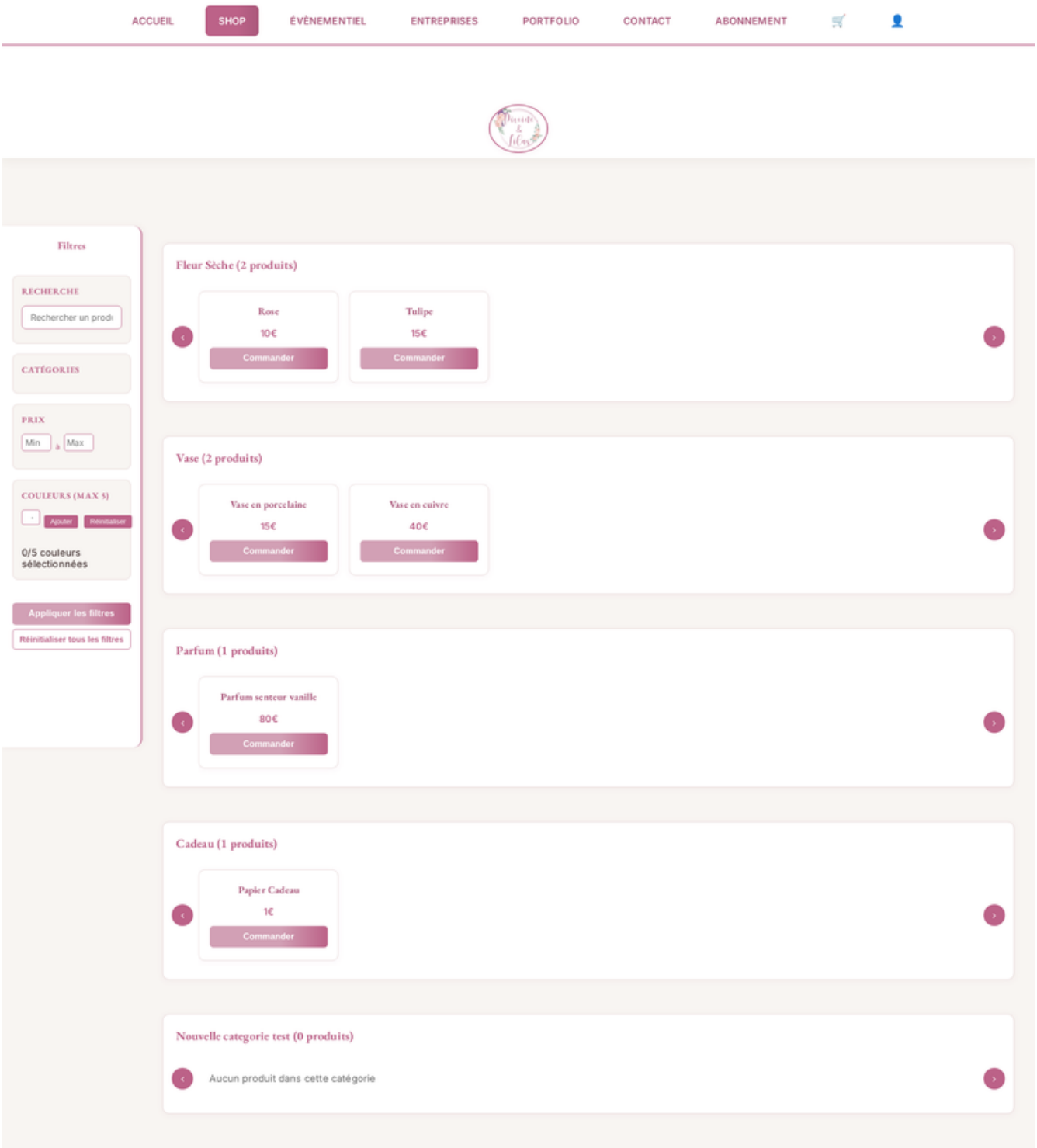


Fig. 5 : Boutique - Desktop

Fig. 6 : Boutique - Mobile

Page panier — Articles avec quantites modifiables, prix unitaire et total.

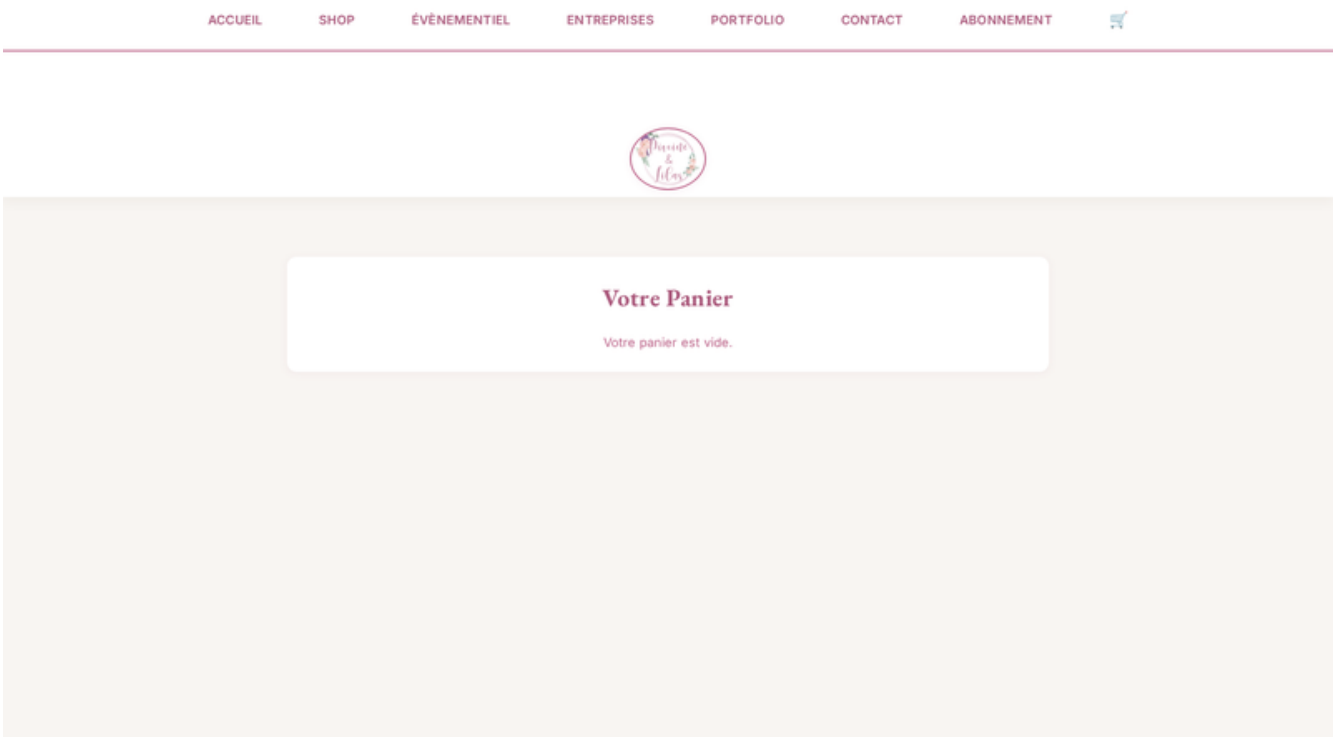


Fig. 7 : Panier - Desktop

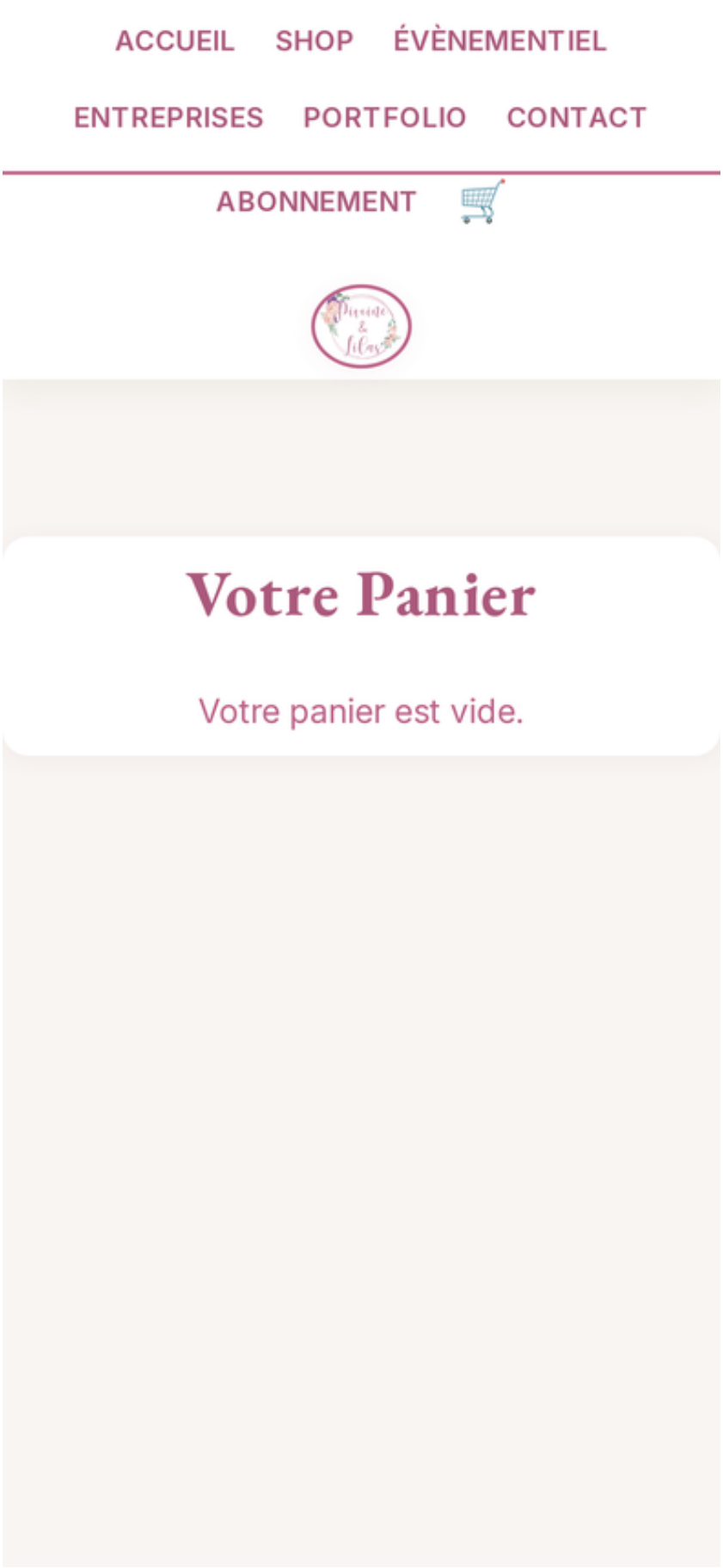


Fig. 8 : Panier - Mobile

Page de paiement Stripe — Formulaire Stripe Elements, resume de commande.

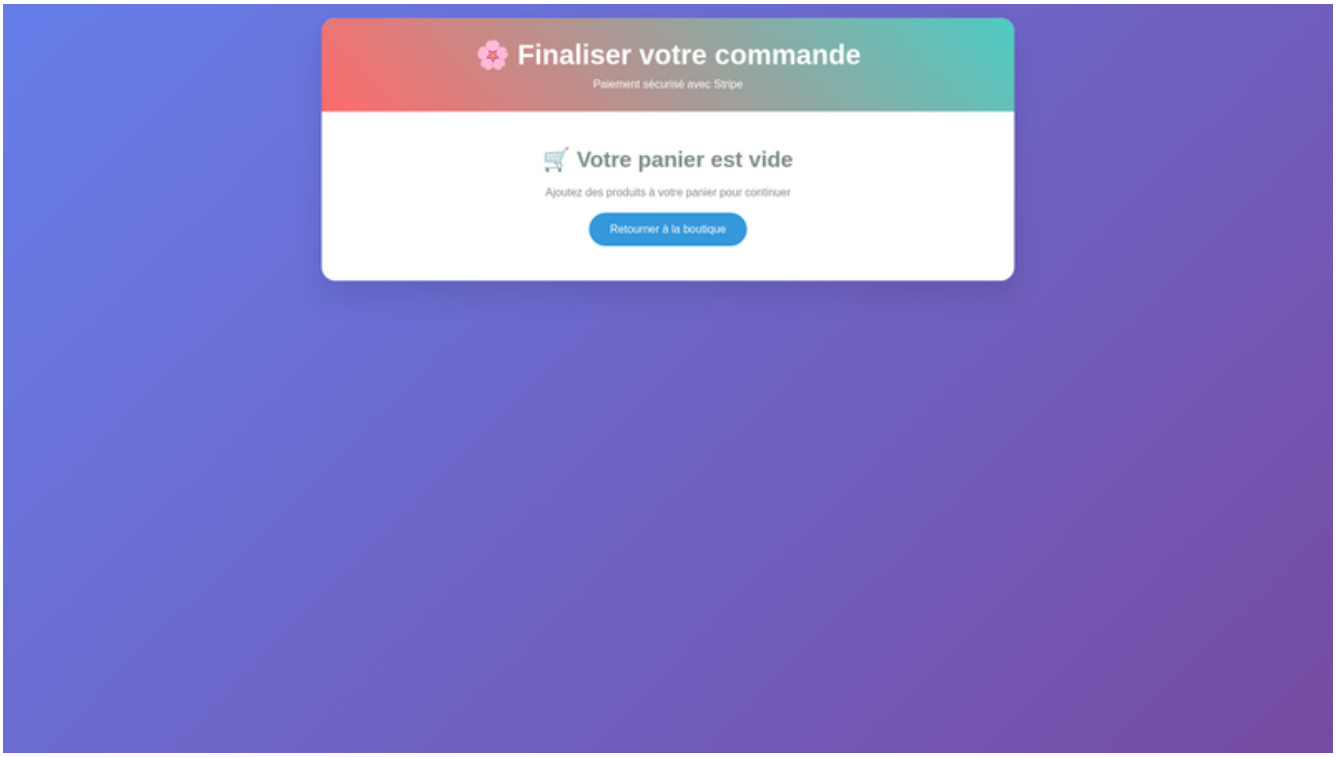


Fig. 9 : Checkout - Desktop

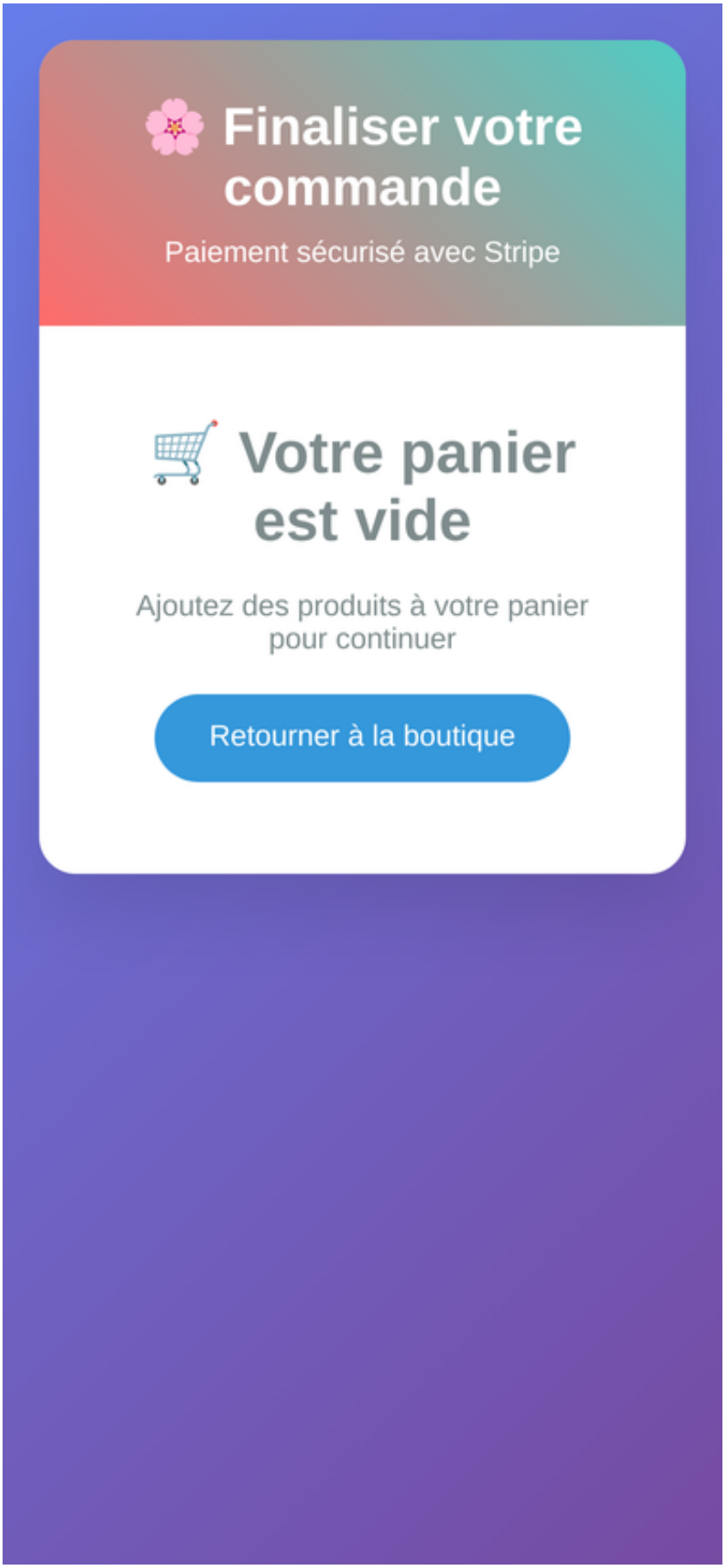


Fig. 10 : Checkout - Mobile

Documentation API Swagger UI — Endpoints classes par namespace (Auth, Products, Categories, Users, Payments).

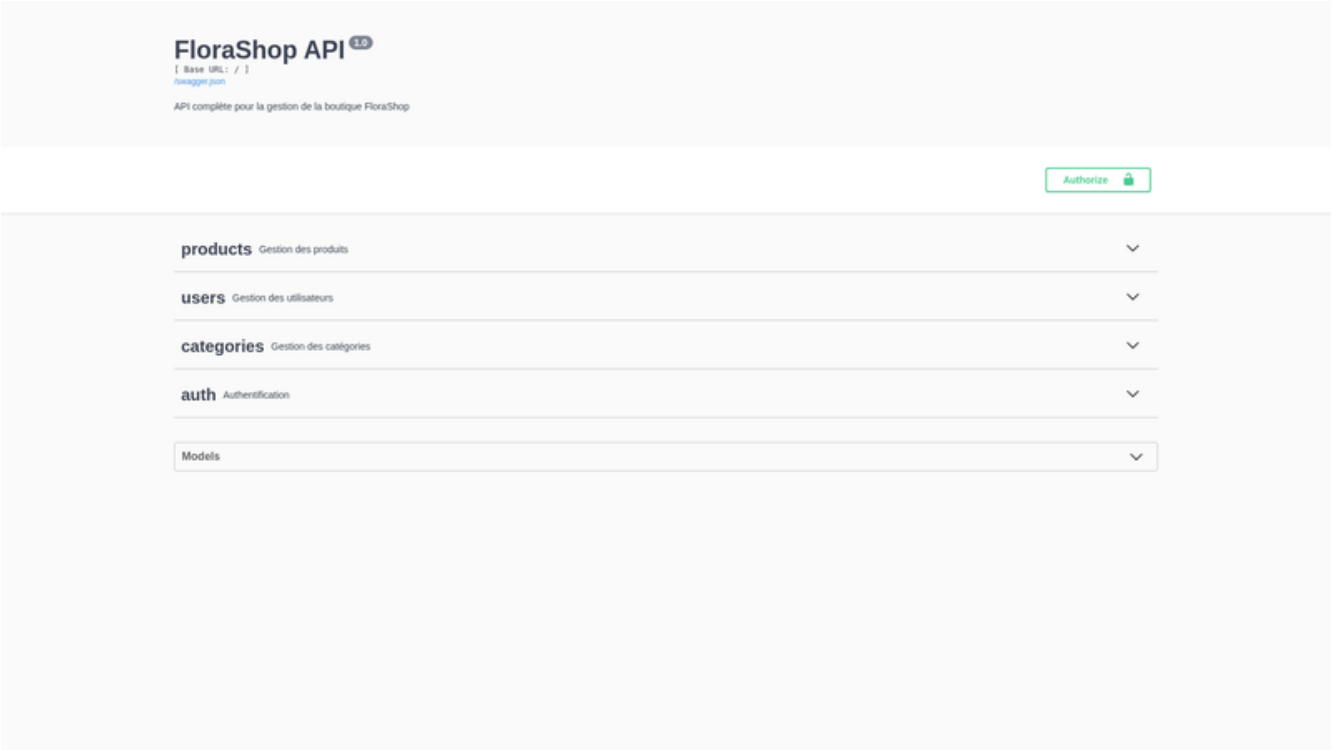


Fig. 11 : Swagger UI - Desktop

Argumentation responsive :

- Les captures montrent le meme contenu adapte a 2 resolutions (desktop 1280px et mobile 393px)
- La navigation s adapte avec des espacements et tailles de police reduites sur mobile (`gap: 7px`, `font-size: 15px`)
- La grille de produits passe de 3 colonnes a 1 colonne sur mobile (media query `@media max-width: 600px`)
- Le formulaire de paiement Stripe s adapte automatiquement a la largeur de l ecran
- Le panier conserve toutes les fonctionnalites en version mobile (modification quantite, suppression)

Extraits de code d interface utilisateur statique

```html
<!-- shop.html — Structure de la page boutique avec navigation responsive -->
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Boutique Florale</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```html
  <link rel="stylesheet" href="../static/css/style.css">
</head>
<body>
  <header>
    <img class="logo" src="..." alt="Logo pivoine & lilas, artisan fleuriste">
    <nav>
      <ul>
        <li><a href="accueil.html">ACCUEIL</a></li>
        <li><a href="shop.html" class="active">SHOP</a></li>
        <li><a href="evenementiel.html">EVENEMENTIEL</a></li>
        <li><a href="entreprises.html">ENTREPRISES</a></li>
        <li><a href="portfolio.html">PORTFOLIO</a></li>
        <li><a href="contact.html">CONTACT</a></li>
        <li><a href="subscription.html">ABONNEMENT</a></li>
        <li><a href="panier.html" id="cart-link">Panier
          <span id="cart-count">0</span></a>
        </li>
        <li id="admin-access" style="display:none;">
          <a href="admin.html">Admin</a>
        </li>
      </ul>
    </nav>
  </header>
  <div class="shop-container">
    <aside class="sidebar-filters">
      <h2>Filtres</h2>
      <input type="text" placeholder="Rechercher un produit..." id="searchInput">
      <div id="categories-filters">
        <!-- Categories chargees dynamiquement depuis l API -->
      </div>
      <input type="number" placeholder="Min" id="minPrice">
      <input type="number" placeholder="Max" id="maxPrice">
    </aside>
    <main class="product-grid" id="productGrid">
      <!-- Produits charges dynamiquement -->
    </main>
  </div>
</body>
</html>
```

```css
/* style.css — Variables CSS et design vintage de la marque Pivoine & Lilas */
@import
↪  url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=EB+Garamond:wght@400;700&di

:root {
    --vintage-bg: #fff6f2;
    --vintage-main: #bc6288;
    --vintage-main-dark: #ab597c;
    --vintage-accent: #d2a0b5;
    --vintage-text: #231111;
    --vintage-card: #fff;
    --vintage-border: #e7d6d6;
    --vintage-shadow: 0 2px 12px rgba(188,98,136,0.07);
}
```

```
body {
    background: var(--vintage-bg);
    color: var(--vintage-text);
    font-family: 'EB Garamond', 'Inter', serif;
    font-size: 18px;
}

nav {
    position: fixed;
    top: 0;
    width: 100%;
    z-index: 1000;
    background: var(--vintage-card);
    border-bottom: 2px solid var(--vintage-main);
    height: 90px;
    display: flex;
    align-items: center;
    justify-content: center;
}

nav a {
    color: var(--vintage-main-dark);
    font-family: 'EB Garamond', serif;
    font-weight: 600;
    font-size: 19px;
    padding: 13px 24px;
    border-radius: 8px;
    transition: background 0.18s, color 0.18s;
}
```

Argumentation :

- **Charte graphique coherente** : utilisation de variables CSS (`:root`) pour garantir l uniformite des couleurs sur toutes les pages. La palette vintage (#bc6288 rose, #fff6f2 fond creme) correspond a l univers fleuri de la marque.
- **Responsive design** : balise `<meta name="viewport">` presente sur toutes les pages, navigation fixe adaptee aux differentes tailles d ecran.
- **Accessibilite** : attributs `alt` sur les images, contrastes de couleurs suffisants (texte sombre #231111 sur fond clair #fff6f2).
- **Separation des responsabilites** : HTML semantique (header, nav, aside, main) separe de la mise en forme CSS dans des fichiers dedies (style.css, account.css, subscription.css).
- **Polices web** : import Google Fonts (EB Garamond pour le style editorial, Inter pour la lisibilite).

CSS responsive — media queries

```
/* style.css — Media queries pour l adaptation tablette et mobile */
@media (max-width: 900px) {
    .shop-container {
        flex-direction: column;  /* Sidebar passe au-dessus des produits */
```

```
        gap: 0;
    }
    .sidebar-filters {
        position: static;        /* Plus de sidebar fixe */
        width: 100%;
        height: auto;
        border-radius: 0 0 12px 12px;
        box-shadow: none;
        border-right: none;
        margin-bottom: 2rem;
    }
    .products-area {
        padding: 1rem 2vw;
    }
    .products-grid {
        grid-template-columns: 1fr;  /* 1 produit par ligne sur tablette */
        gap: 1.2rem;
    }
}

@media (max-width: 600px) {
    .main-content {
        padding: 8px 2vw 14px 2vw;  /* Marges reduites sur mobile */
    }
    nav ul {
        gap: 7px;                    /* Navigation compacte */
    }
    nav a {
        padding: 8px 7px;
        font-size: 15px;             /* Taille reduite pour petit ecran */
    }
    .logo {
        width: 55px;
        height: 40px;
        margin: 60px auto 6px auto;  /* Logo redimensionne */
    }
}
```

Argumentation responsive :

- **Mobile-first implicite** : les media queries ciblent les breakpoints 900px (tablette) et 600px (mobile). En dessous de 900px, le layout passe de 2 colonnes (sidebar + grille) a 1 colonne empilee.
- **Grille CSS adaptative** : `grid-template-columns: repeat(auto-fill, minmax(260px, 1fr))` sur desktop permet un nombre variable de colonnes selon la largeur, puis se reduit a `1fr` (1 colonne) sur mobile.
- **Navigation responsive** : espacement et taille de police reduits sur mobile (`gap: 7px`, `font-size: 15px`) pour que tous les liens restent accessibles sans debordement.

Formulaire securise — inscription avec reCAPTCHA

```html
<!-- register.html — Formulaire d inscription avec validation et protection anti-bot -->
<form class="login-form" id="register-form">
    <h1>Creation de compte</h1>
    <div class="form-group">
        <label for="username">Nom d utilisateur</label>
        <input type="text" id="username" required>
    </div>
    <div class="form-group">
        <label for="email">Email</label>
        <input type="email" id="email" required>  <!-- Validation HTML5 du format email -->
    </div>
    <div class="form-group">
        <label for="password">Mot de passe</label>
        <input type="password" id="password" required>
    </div>
    <div class="form-group">
        <label for="confirm-password">Confirmer le mot de passe</label>
        <input type="password" id="confirm-password" required>
    </div>
    <div class="form-group">
        <!-- Protection anti-bot Google reCAPTCHA -->
        <div class="g-recaptcha" data-sitekey="YOUR_SITE_KEY"></div>
    </div>
    <button type="submit" class="login-button">Creer mon compte</button>
</form>

<script>
    document.getElementById('register-form').addEventListener('submit', async function(e) {
        e.preventDefault();
        const password = document.getElementById('password').value;
        const confirmPassword = document.getElementById('confirm-password').value;
        // Validation cote client : verification de correspondance des mots de passe
        if (password !== confirmPassword) {
            alert('Les mots de passe ne correspondent pas');
            return;
        }
        const result = await apiService.register(
            document.getElementById('username').value,
            document.getElementById('email').value,
            password
        );
        if (result.success) {
            alert('Compte cree avec succes !');
            window.location.href = 'accueil.html';
        } else {
            alert(result.error || 'Erreur lors de la creation du compte');
        }
    });
</script>
```

Argumentation securite front-end :

- **Validation HTML5 native** : les attributs `type="email"` et `required` empechent la soumission de formulaires avec des champs vides ou un format d email invalide, avant

meme que le JavaScript ne s execute.

- **Confirmation de mot de passe** : la verification `password !== confirmPassword` cote client empeche les erreurs de saisie.
- **reCAPTCHA** : le widget Google reCAPTCHA protege le formulaire contre les inscriptions automatisees (bots). La cle publique est chargee via `<script src="https://www.google.com/re` avec les attributs `async defer` pour ne pas bloquer le rendu de la page.
- **Echappement Jinja2** : sur toutes les pages, les variables affichees dans les templates Jinja2 sont automatiquement echappees par le moteur de templates, protegeant contre les attaques XSS (Cross-Site Scripting).

Extraits de code de la partie dynamique des interfaces utilisateur

```
// api.js — Classe ApiService gerant les appels API et l authentification cote client
const API_BASE_URL = 'http://localhost:5000/api/v1';

class ApiService {
    constructor() {
        this.token = localStorage.getItem('auth_token');
        this.user = JSON.parse(localStorage.getItem('user') || 'null');
        this.updateProfileUI();
    }

    getHeaders() {
        const headers = { 'Content-Type': 'application/json' };
        if (this.token) {
            headers['Authorization'] = `Bearer ${this.token}`;
        }
        return headers;
    }

    async login(email, password) {
        const response = await fetch(`${API_BASE_URL}/auth/login`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
            body: JSON.stringify({ email, password })
        });
        const data = await response.json();
        if (response.ok && data.success) {
            this.user = data.data.user;
            this.token = data.data.token;
            localStorage.setItem('auth_token', this.token);
            localStorage.setItem('user', JSON.stringify(this.user));
            this.updateProfileUI();
            return { success: true, data: data.data };
        }
        return { success: false, error: data.message || 'Erreur de connexion' };
    }

    async register(username, email, password) {
        const response = await fetch(`${API_BASE_URL}/auth/register`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
            body: JSON.stringify({ username, email, password })
```

```
        });
        const data = await response.json();
        if (response.ok && data.success) {
            this.user = data.data.user;
            localStorage.setItem('user', JSON.stringify(this.user));
            this.updateProfileUI();
            return { success: true, data: data.data };
        }
        return { success: false, error: data.message || 'Erreur lors de la creation du
        ↪  compte' };
    }

    logout() {
        this.token = null;
        this.user = null;
        localStorage.removeItem('auth_token');
        localStorage.removeItem('user');
        this.updateProfileUI();
    }

    // Gestion des produits — appels CRUD vers l API
    async getProducts() {
        const response = await fetch(`${API_BASE_URL}/products`, { headers: this.getHeaders()
        ↪  });
        return await response.json();
    }

    async getCategories() {
        const response = await fetch(`${API_BASE_URL}/categories`, { headers:
        ↪  this.getHeaders() });
        return await response.json();
    }
}
```

Argumentation :

- **Architecture orientee objet** : la classe `ApiService` encapsule toute la logique d appel API, la gestion du token JWT et la mise a jour de l interface utilisateur.
- **Authentification cote client** : le token JWT est stocke dans le `localStorage` et envoye automatiquement dans le header `Authorization: Bearer` de chaque requete.
- **Gestion de session** : les methodes `login()`, `register()`, `logout()` gerent l etat de l utilisateur et mettent a jour l interface (affichage du bouton connexion/deconnexion, email utilisateur, acces admin).
- **Appels API asynchrones** : utilisation de `async/await` avec `fetch()` pour des appels non-bloquants et une gestion propre des erreurs.
- **Separation front-end / back-end** : le front-end communique exclusivement via l API REST, ce qui garantit une architecture decouplable.

Gestion dynamique du panier (JavaScript)

```
// panier.html — Fonctions de gestion du panier cote client
function renderCart() {
    const cart = JSON.parse(localStorage.getItem('cart') || '[]');
    const cartContent = document.getElementById('cart-content');
    const cartSummary = document.getElementById('cart-summary');
    const cartTotal = document.getElementById('cart-total');

    if (cart.length === 0) {
        cartContent.innerHTML = "<p>Votre panier est vide.</p>";
        cartSummary.style.display = "none";
        return;
    }

    let total = 0;
    cartContent.innerHTML = `
        <table class="cart-table">
          <thead>
            <tr><th>Produit</th><th>Categorie</th><th>Prix</th><th>Retirer</th></tr>
          </thead>
          <tbody>
            ${cart.map((prod, idx) => {
              total += Number(prod.price);
              return `
                <tr>
                  <td>${prod.name}</td>
                  <td>${prod.category}</td>
                  <td>${Number(prod.price).toFixed(2)} EUR</td>
                  <td><button class="remove-btn"
↪   onclick="removeFromCart(${idx})">X</button></td>
                </tr>`;
            }).join('')}
          </tbody>
        </table>`;
    cartTotal.textContent = "Total : " + total.toFixed(2) + " EUR";
    cartSummary.style.display = "flex";
}

function removeFromCart(idx) {
    let cart = JSON.parse(localStorage.getItem('cart') || '[]');
    cart.splice(idx, 1);
    localStorage.setItem('cart', JSON.stringify(cart));
    renderCart();
    updateCartCount();
}

function updateCartCount() {
    const cart = JSON.parse(localStorage.getItem('cart') || '[]');
    const badge = document.getElementById('cart-count');
    if (badge) {
        badge.textContent = cart.length;
        badge.style.display = cart.length > 0 ? 'inline-block' : 'none';
    }
}
```

Argumentation :

- **Persistance locale** : le panier est stocke dans le `localStorage` du navigateur sous forme de tableau JSON. L utilisateur conserve son panier meme apres fermeture du navigateur (sans cookies serveur).
- **Rendu dynamique du DOM** : la fonction `renderCart()` reconstruit le tableau HTML a chaque modification, utilisant `Array.map()` et les template literals ES6 pour generer les lignes du tableau.
- **Mise a jour en temps reel** : le compteur du badge panier (`cart-count`) est mis a jour a chaque ajout ou suppression via `updateCartCount()`.
- **Manipulation securisee** : `splice(idx, 1)` supprime l element a l index exact sans affecter les autres elements du panier.

Integration du paiement Stripe.js (checkout)

```
// checkout.html — Integration Stripe.js v3 pour le paiement securise
let stripe = null;
let elements = null;
let card = null;

async function initializeStripe() {
    // Appel API pour creer un Payment Intent et obtenir la cle publique Stripe
    const response = await fetch('/api/v1/payments/create-payment-intent', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            items: cartItems.map(item => ({
                product_id: item.product_id,
                quantity: item.quantity
            })),
            email: 'temp@example.com'
        })
    });

    if (response.ok) {
        const data = await response.json();
        // Initialisation de Stripe avec la cle publique retournee par le serveur
        stripe = Stripe(data.stripe_publishable_key);
        elements = stripe.elements();
        card = elements.create('card', {
            style: {
                base: { fontSize: '16px', color: '#424770',
                        '::placeholder': { color: '#aab7c4' } }
            }
        });
        card.mount('#card-element');

        // Affichage des erreurs de saisie en temps reel
        card.on('change', ({error}) => {
            const displayError = document.getElementById('card-errors');
            displayError.textContent = error ? error.message : '';
            displayError.style.display = error ? 'block' : 'none';
        });
    }
```

```
}

// Soumission du formulaire de paiement
document.getElementById('payment-form').addEventListener('submit', async (event) => {
    event.preventDefault();
    const email = document.getElementById('email').value;
    const name = document.getElementById('name').value;

    // Creer le Payment Intent definitif
    const response = await fetch('/api/v1/payments/create-payment-intent', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            items: cartItems.map(item => ({
                product_id: item.product_id, quantity: item.quantity
            })),
            email: email
        })
    });
    const { client_secret, order_id } = await response.json();

    // Confirmation du paiement via Stripe.js
    const {error, paymentIntent} = await stripe.confirmCardPayment(client_secret, {
        payment_method: {
            card: card,
            billing_details: { name: name, email: email }
        }
    });

    if (error) {
        // Affichage de l erreur de paiement
        document.getElementById('payment-result').innerHTML =
            `<div class="error"><h3>Erreur de paiement</h3><p>${error.message}</p></div>`;
    } else {
        // Paiement reussi : affichage de la confirmation
        document.getElementById('payment-result').innerHTML =
            `<div class="success"><h3>Paiement reussi !</h3>
             <p>Commande #${order_id} confirmee</p>
             <p>Transaction: ${paymentIntent.id}</p></div>`;
        localStorage.removeItem('cart');  // Vider le panier
    }
});
```

Argumentation securite paiement :

- **Delegation a Stripe.js** : les coordonnees bancaires (numero de carte, date, CVC) ne transitent jamais par notre serveur. Le composant `card` de Stripe Elements les envoie directement aux serveurs Stripe via un iframe securise (conformite PCI DSS).
- **Flux Payment Intent** : le `client_secret` est genere cote serveur avec un montant recalcule a partir des prix en base (pas de montant venant du client). Stripe confirme le paiement via ce secret unique.
- **Validation en temps reel** : `card.on('change')` affiche les erreurs de saisie de carte instantanement, sans requete serveur.

- **Nettoyage post-paiement** : apres un paiement reussi, le panier est vide (`localStorage.removeItem('cart')`) et le formulaire est masque pour empecher un double paiement.
- **Gestion des erreurs** : les erreurs Stripe (carte refusee, fonds insuffisants...) sont affichees a l utilisateur sans exposer de details techniques sensibles.

Elements significatifs cote back-end

Architecture technique de l application

```
┌──────────────────────────────────────────────────────────────────────┐
│                        NAVIGATEUR CLIENT                             │
│  HTML5 / CSS3 / JavaScript (api.js, auth.js) / Stripe.js v3          │
└──────────────────────────────────────────────────────────────────────┘
                              │ HTTP / HTTPS (JSON)
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        SERVEUR FLASK (Python 3.10)                   │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐       │
│  │ Flask-RESTX  │  │   Jinja2     │  │   Flask-Migrate      │       │
│  │ (API REST)   │  │ (Templates)  │  │   (Alembic)          │       │
│  └──────────────┘  └──────────────┘  └──────────────────────┘       │
│         │                 │                                          │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │                Services metier                           │       │
│  │ StripeService │ Facade │ Auth (PyJWT)                    │       │
│  └──────────────────────────────────────────────────────────┘       │
│         │                 │                                          │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │              Repositories (Pattern Repository)            │       │
│  │ CategoryRepo │ ProductRepo │ UserRepo │ PriceRepo         │       │
│  └──────────────────────────────────────────────────────────┘       │
│                              │                                       │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │             SQLAlchemy ORM (Models)                       │       │
│  │ User │ Product │ Category │ Order │ OrderItem │ ...       │       │
│  └──────────────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────────────┘
                              │ SQL (psycopg2)
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                POSTGRESQL (base florashop)                           │
│  7 tables : users, categories, products, orders,                     │
```

```
│              order_items, reviews, prices                          │
└────────────────────────────────────────────────────────────────────┘

           Integrations externes :
           ┌──────────────────────────┐
           │     API Stripe           │
           │  Payment Intents         │
           │  Webhooks (signatures)   │
           └──────────────────────────┘
```

Argumentation :

- **Architecture 3-tiers** : la separation entre le navigateur (presentation), le serveur Flask (logique metier) et PostgreSQL (donnees) garantit la maintenabilite et la scalabilite.
- **Couches internes du serveur** : les routes (Flask-RESTX) delegent aux services metier, qui delegent aux repositories, qui utilisent l ORM SQLAlchemy. Cette architecture en couches facilite les tests et la reutilisation du code.
- **Integration externe** : l API Stripe est appelee uniquement depuis le `StripeService` cote serveur (jamais directement depuis le client), sauf Stripe.js qui communique directement avec Stripe pour les donnees bancaires (conformite PCI DSS).

Presentation de la base de donnees

Schema conceptuel (MCD) - donnees et relations

Le MCD suit la methode Merise. Les 7 entites du systeme et leurs associations sont representees ci-dessous.

Entites :

Entite	Attributs	Identifiant
USERS	username (VARCHAR 50, unique, NN), email (VARCHAR 120, unique, NN), password (VARCHAR 255, NN), is_admin (BOOL, defaut FALSE)	id (INT, PK)
CATEGORIES	name (VARCHAR 100, unique, NN), created_at (DATETIME)	id (INT, PK, auto)
PRODUCTS	name (VARCHAR 255, NN), price (FLOAT, NN), is_on_sale (BOOL, defaut FALSE)	id (INT, PK)

Entite	Attributs	Identifiant
ORDERS	email (VARCHAR 120, NN), total_amount (FLOAT, NN), stripe_payment_intent_id (VARCHAR 255, nullable), status (VARCHAR 50, defaut ‘pending’), created_at (DATETIME)	id (INT, PK, auto)
ORDER_ITEMS	quantity (INT, NN, defaut 1), price (FLOAT, NN — prix au moment de l achat)	id (INT, PK, auto)
REVIEWS	content (TEXT, NN), rating (INT, NN)	id (INT, PK)
PRICES	amount (FLOAT, NN), is_active (BOOL, defaut TRUE), created_at (DATETIME), updated_at (DATETIME)	id (INT, PK)

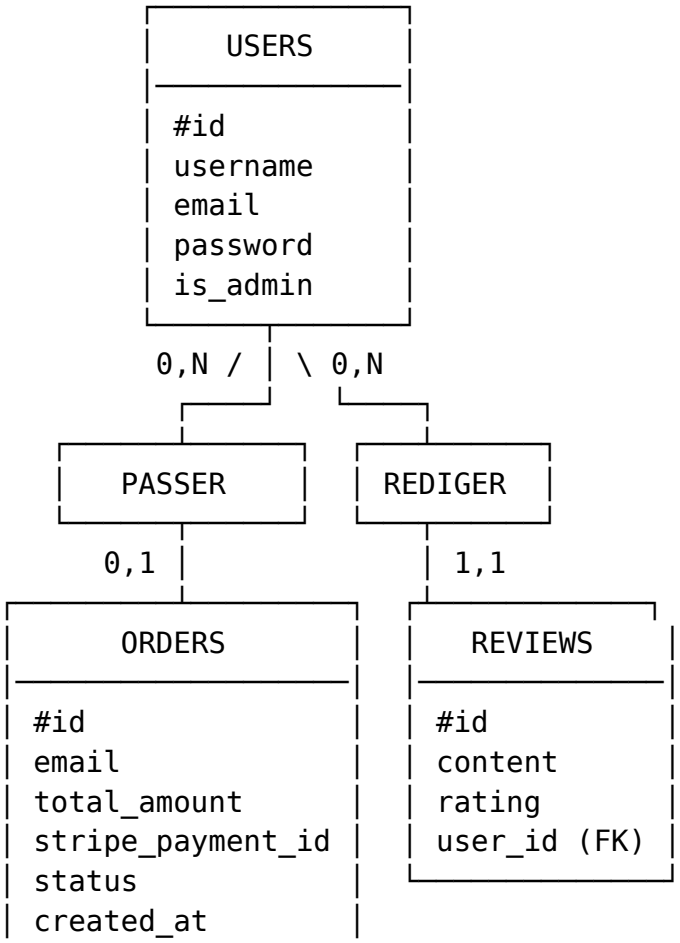
Associations et cardinalites (notation Merise) :

Association	Entite 1	Cardinalite	Entite 2	Cardinalite	Description
PASSER	USERS	0,N	ORDERS	0,1	Un utilisateur peut passer 0 a N commandes. Une commande peut etre passee par un utilisateur (ou en invite, d ou 0,1)

Association	Entite 1	Cardinalite	Entite 2	Cardinalite	Description
COMPOSER	ORDERS	1,N	ORDER_ITEMS	1,1	Une commande contient au moins 1 ligne de commande. Chaque ligne appartient a exactement 1 commande (cascade delete-orphan)
REFERENCER	ORDER_ITEMS	1,1	PRODUCTS	0,N	Chaque ligne de commande reference exactement 1 produit. Un produit peut apparaitre dans 0 a N lignes de commande
CONTENIR	CATEGORIES	1,N	PRODUCTS	1,1	Une categorie contient 1 a N produits. Chaque produit appartient a exactement 1 categorie (cascade delete)

Association	Entite 1	Cardinalite	Entite 2	Cardinalite	Description
REDIGER	USERS	0,N	REVIEWS	1,1	Un utilisateur peut rediger 0 a N avis. Chaque avis est redige par exactement 1 utilisateur
HISTORISER	PRODUCTS	0,N	PRICES	1,1	Un produit peut avoir 0 a N prix (historique). Chaque prix appartient a exactement 1 produit

Diagramme entites-relations :



```
│ user_id (FK)      │
└──────────────────┘
      1,N │
   ┌──────────────┐
   │  COMPOSER    │
   └──────────────┘
      1,1 │
┌──────────────────────────┐
│     ORDER_ITEMS          │
├──────────────────────────┤
│ #id                      │
│ quantity                 │
│ price                    │
│ order_id (FK)            │
│ product_id (FK)          │
└──────────────────────────┘
      1,1 │
   ┌──────────────┐
   │ REFERENCER   │
   └──────────────┘
      0,N │
┌──────────────────────────┐
│     PRODUCTS             │
├──────────────────────────┤
│ #id                      │
│ name                     │
│ price                    │
│ category_id (FK)         │
│ is_on_sale               │
└──────────────────────────┘
      0,N │
   ┌──────────────┐
   │  HISTORISER  │
   └──────────────┘
      1,1 │
┌──────────────────────────┐
│      PRICES              │
├──────────────────────────┤
│ #id                      │
│ amount                   │
│ is_active                │
│ product_id (FK)          │
│ created_at               │
│ updated_at               │
└──────────────────────────┘
```

```
┌──────────────────┐
│  CATEGORIES      │
├──────────────────┤
│ #id              │
│ name             │
│ created_at       │
└──────────────────┘
      1,N │
┌──────────────────┐
│    CONTENIR      │
└──────────────────┘
      1,1 │
```

Regles de gestion : - RG1 : Un utilisateur est identifie par un couple (username, email) unique - RG2 : Une commande peut etre passee par un utilisateur connecte ou un invite (user_id nullable) - RG3 : Le prix dans ORDER_ITEMS est fige au moment de l achat (independant des modifications ulterieures du produit) - RG4 : La suppression d une categorie entraine la suppression en cascade de tous ses produits - RG5 : La suppression d une commande entraine la suppression en cascade de toutes ses lignes (delete-orphan) - RG6 : L historique des prix est conserve via la table PRICES (seul le prix avec is_active=TRUE est le prix courant) - RG7 : Le statut d une commande suit le cycle de vie : pending → paid | failed | cancelled

Schema physique (MLD)

Table	Colonnes	Cles
users	id (INT PK), username (VARCHAR 50 UNIQUE NN), email (VARCHAR 120 UNIQUE NN), password (VARCHAR 255 NN), is_admin (BOOL default FALSE)	PK : id
categories	id (INT PK), name (VARCHAR 100 UNIQUE NN), created_at (DATETIME)	PK : id
products	id (INT PK), name (VARCHAR 255 NN), price (FLOAT NN), category_id (INT NN), is_on_sale (BOOL default FALSE)	PK : id, FK : category_id → categories.id (CASCADE)
orders	id (INT PK AUTO), user_id (INT NULL), email (VARCHAR 120 NN), total_amount (FLOAT NN), stripe_payment_intent_id (VARCHAR 255 NULL), status (VARCHAR 50 default ‘pending’), created_at (DATETIME)	PK : id, FK : user_id → users.id
order_items	id (INT PK AUTO), order_id (INT NN), product_id (INT NN), quantity (INT NN default 1), price (FLOAT NN)	PK : id, FK : order_id → orders.id, FK : product_id → products.id

Table	Colonnes	Cles
reviews	id (INT PK), content (TEXT NN), rating (INT NN), user_id (INT NN)	PK : id, FK : user_id → users.id
prices	id (INT PK), amount (FLOAT NN), is_active (BOOL default TRUE), product_id (INT NN), created_at (DATETIME), updated_at (DATETIME)	PK : id, FK : product_id → products.id

Script de creation ou modification de la base de donnees

```sql
-- Structure de la base de donnees FloraShop (PostgreSQL)
-- 7 tables correspondant au MLD ci-dessus

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    is_admin BOOLEAN DEFAULT FALSE
);

CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    price FLOAT NOT NULL,
    category_id INTEGER NOT NULL REFERENCES categories(id) ON DELETE CASCADE,
    is_on_sale BOOLEAN DEFAULT FALSE
);

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),  -- nullable pour les invites
    email VARCHAR(120) NOT NULL,
    total_amount FLOAT NOT NULL,
    stripe_payment_intent_id VARCHAR(255),
    status VARCHAR(50) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
```

```sql
    product_id INTEGER NOT NULL REFERENCES products(id),
    quantity INTEGER NOT NULL DEFAULT 1,
    price FLOAT NOT NULL  -- prix fige au moment de l achat
);

CREATE TABLE reviews (
    id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    rating INTEGER NOT NULL,
    user_id INTEGER NOT NULL REFERENCES users(id)
);

CREATE TABLE prices (
    id SERIAL PRIMARY KEY,
    amount FLOAT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    product_id INTEGER NOT NULL REFERENCES products(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insertion des donnees initiales
INSERT INTO categories (name) VALUES
    ('Fleurs Fraiches'), ('Vases'), ('Parfums'),
    ('Plantes d Interieur'), ('Accessoires'), ('Compositions'), ('Cadeaux');
```

Modeles ORM (SQLAlchemy) — correspondance avec le schema physique

```python
# app/models/user.py — Modele utilisateur avec securite

class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)
    is_admin = db.Column(db.Boolean, default=False)

    def to_dict(self):
        """Serialisation securisee : le mot de passe est EXCLU du dictionnaire"""
        return {
            'id': self.id,
            'username': self.username,
            'email': self.email,
            'is_admin': self.is_admin
            # password volontairement absent
        }
```

```python
# app/models/product.py — Modele produit avec serialisation defensive

class Product(db.Model):
    __tablename__ = 'products'
    id = db.Column(db.Integer, primary_key=True)
```

```
    name = db.Column(db.String(255), nullable=False)
    price = db.Column(db.Float, nullable=False)
    category_id = db.Column(db.Integer, db.ForeignKey('categories.id'), nullable=False)
    is_on_sale = db.Column(db.Boolean, default=False)
    category = db.relationship('Category', back_populates='products')

    def to_dict(self):
        """Conversion en dictionnaire avec chargement de la categorie associee"""
        result = {
            'id': int(self.id) if self.id is not None else None,
            'name': str(self.name) if self.name else '',
            'price': float(self.price) if self.price is not None else 0.0,
            'category_id': int(self.category_id) if self.category_id is not None else None,
            'is_on_sale': bool(self.is_on_sale)
        }
        if hasattr(self, 'category') and self.category:
            result['category'] = { 'id': int(self.category.id), 'name':
↪   str(self.category.name) }
        return result
```

```
# app/models/order.py — Modeles Order et OrderItem geres par SQLAlchemy ORM

class Order(db.Model):
    __tablename__ = 'orders'
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=True)
    email = db.Column(db.String(120), nullable=False)
    total_amount = db.Column(db.Float, nullable=False)
    stripe_payment_intent_id = db.Column(db.String(255), nullable=True)
    status = db.Column(db.String(50), default='pending')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    order_items = db.relationship('OrderItem', backref='order', lazy=True,
                                  cascade='all, delete-orphan')

class OrderItem(db.Model):
    __tablename__ = 'order_items'
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    order_id = db.Column(db.Integer, db.ForeignKey('orders.id'), nullable=False)
    product_id = db.Column(db.Integer, db.ForeignKey('products.id'), nullable=False)
    quantity = db.Column(db.Integer, nullable=False, default=1)
    price = db.Column(db.Float, nullable=False)
    product = db.relationship('Product', backref='order_items')
```

Argumentation :

- **Coherence MCD → MLD → ORM** : chaque table du script SQL correspond exactement a un modele SQLAlchemy. Les types SERIAL PostgreSQL correspondent aux `db.Integer` avec `primary_key=True` et `autoincrement=True`.
- **Contraintes d integrite** : les REFERENCES SQL et les `db.ForeignKey` Python assurent l integrite referentielle. Les clauses `ON DELETE CASCADE` sur `products` et `order_items` correspondent aux `cascade='all, delete-orphan'` de SQLAlchemy.
- **Securite du modele User** : la methode `to_dict()` exclut volontairement le champ `password` de la serialisation pour empecher toute fuite de mot de passe via l API.

- **Serialisation defensive** : le `Product.to_dict()` utilise des conversions explicites (`int()`, `float()`, `str()`) et des verifications `is not None` pour eviter les erreurs de serialisation sur des objets partiellement charges.
- **Gestion des invites** : `user_id` est nullable dans `orders` (`nullable=True`), permettant les commandes sans connexion.

```
# app/services/stripe_service.py — Service metier de paiement Stripe

class StripeService:
    def __init__(self):
        stripe.api_key = current_app.config['STRIPE_SECRET_KEY']

    def create_payment_intent(self, order_data):
        """Cree un Payment Intent Stripe pour une commande"""
        # Calculer le montant total
        total_amount = 0
        order_items = []
        for item in order_data['items']:
            product = Product.query.get(item['product_id'])
            if not product:
                raise ValueError(f"Produit {item['product_id']} non trouve")
            quantity = item['quantity']
            total_amount += product.price * quantity
            order_items.append({
                'product_id': product.id, 'quantity': quantity, 'price': product.price
            })

        # Creer la commande en base
        order = Order(
            user_id=order_data.get('user_id'),
            email=order_data['email'],
            total_amount=total_amount, status='pending'
        )
        db.session.add(order)
        db.session.flush()

        for item_data in order_items:
            db.session.add(OrderItem(order_id=order.id, **item_data))

        # Creer le Payment Intent chez Stripe
        intent = stripe.PaymentIntent.create(
            amount=int(total_amount * 100),  # Stripe utilise les centimes
            currency='eur',
            metadata={'order_id': order.id, 'email': order_data['email']},
            automatic_payment_methods={'enabled': True}
        )
        order.stripe_payment_intent_id = intent.id
        db.session.commit()

        return {
            'client_secret': intent.client_secret,
            'order_id': order.id,
            'total_amount': total_amount,
            'stripe_publishable_key': current_app.config['STRIPE_PUBLISHABLE_KEY']
        }
```

```python
    def handle_webhook(self, payload, sig_header):
        """Gere les webhooks Stripe pour les evenements de paiement"""
        event = stripe.Webhook.construct_event(
            payload, sig_header, current_app.config['STRIPE_WEBHOOK_SECRET']
        )
        if event['type'] == 'payment_intent.succeeded':
            self.confirm_payment(event['data']['object']['id'])
        elif event['type'] == 'payment_intent.payment_failed':
            self.confirm_payment(event['data']['object']['id'])
        return {'status': 'success'}
```

```python
# app/services/facade.py — Facade agregant tous les repositories

from ..persistence import (
    UserRepository, ProductRepository, CategoryRepository,
    ReviewRepository, PriceRepository
)

class Facade:
    """Point d acces unique a toutes les couches d acces aux donnees"""
    def __init__(self):
        self.users = UserRepository()
        self.products = ProductRepository()
        self.categories = CategoryRepository()
        self.reviews = ReviewRepository()
        self.prices = PriceRepository()
```

```python
# app/__init__.py — Factory de l application Flask (extrait)

def create_app():
    app = Flask(__name__)

    # Configuration CORS restrictive (origines autorisees)
    CORS(app, resources={
        r"/api/*": {
            "origins": ["http://localhost:8000", "http://localhost:5000"],
            "methods": ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
            "allow_headers": ["Content-Type", "Authorization"]
        }
    })

    app.config.update(
        SQLALCHEMY_DATABASE_URI = 'postgresql://postgres:root@localhost:5432/florashop',
        SECRET_KEY = os.environ.get('SECRET_KEY', 'dev_secret_key_123'),
        ADMIN_TOKEN = os.environ.get('ADMIN_TOKEN', 'florashop_admin_2024_secure')
    )

    db.init_app(app)
    migrate.init_app(app, db)

    # Swagger UI avec authentification Bearer
    authorizations = {
        'Bearer Auth': {
            'type': 'apiKey', 'in': 'header', 'name': 'Authorization',
```

```
            'description': 'Token JWT retourne par /auth/login'
        }
    }
    api = Api(app, version='1.0', title='FloraShop API',
              doc='/api/v1', authorizations=authorizations, security='Bearer Auth')

    # Enregistrement des blueprints et namespaces
    app.register_blueprint(api_bp, url_prefix='/api/v1')
    app.register_blueprint(payments_bp, url_prefix='/api/v1/payments')
    api.add_namespace(products_ns, path='/api/v1/products')
    api.add_namespace(categories_ns, path='/api/v1/categories')
    api.add_namespace(users_ns, path='/api/v1/users')
    api.add_namespace(auth_ns, path='/api/v1/auth')

    return app
```

Argumentation :

- **Regles de gestion metier** : le `StripeService` encapsule toute la logique de paiement — calcul du total a partir des produits en base (pas de prix venant du client), creation de la commande avant le paiement, mise a jour du statut apres confirmation.
- **Securite financiere** : le montant est recalcule cote serveur a partir des prix en base de donnees (jamais confie au client). La conversion en centimes (`int(total * 100)`) suit les bonnes pratiques Stripe.
- **Gestion transactionnelle** : utilisation de `db.session.flush()` pour obtenir l ID de la commande avant de creer les items, puis `db.session.commit()` en fin de processus. En cas d erreur, `db.session.rollback()` est appele.
- **Verification webhook** : `stripe.Webhook.construct_event()` valide la signature du webhook pour empecher toute falsification d evenement.
- **Pattern Facade** : la classe Facade fournit un point d acces unique a tous les repositories, simplifiant l utilisation depuis les routes et services. Chaque repository gere une entite (Users, Products, Categories, Reviews, Prices).
- **Factory pattern** : `create_app()` centralise la configuration, l initialisation des extensions (SQLAlchemy, Migrate, CORS) et l enregistrement des namespaces API. Le pattern Factory permet de creer plusieurs instances de l application (test, dev, production) avec des configurations differentes.
- **CORS restrictif** : seuls les origines `localhost:5000` et `localhost:8000` sont autorisees, avec des methodes HTTP explicitement listees (`GET, POST, PUT, DELETE, OPTIONS`).
- **Swagger avec authentification** : la configuration `authorizations` ajoute un header `Authorization: Bearer` dans l interface Swagger UI, permettant de tester les endpoints proteges directement depuis la documentation.

Extraits de code de composants d acces aux donnees

```
# app/persistence/category_repository.py — Repository d acces aux donnees des categories

class CategoryRepository:
```

```python
    @staticmethod
    def get_all() -> List[Category]:
        """Recuperer toutes les categories avec validation ID"""
        categories = Category.query.all()
        return [cat for cat in categories if cat.id is not None]

    @staticmethod
    def get_by_id(category_id: int) -> Optional[Category]:
        """Recuperer une categorie par ID avec validation"""
        category = Category.query.get(category_id)
        if category and category.id is None:
            return None
        return category

    @staticmethod
    def create(name: str) -> Optional[Category]:
        """Creer une nouvelle categorie avec validation d unicite"""
        existing = Category.query.filter_by(name=name).first()
        if existing:
            return None
        category = Category(name=name)
        db.session.add(category)
        db.session.flush()
        if category.id is None:
            db.session.rollback()
            return None
        db.session.commit()
        return category

    @staticmethod
    def update(category_id: int, name: str) -> Optional[Category]:
        """Modifier une categorie avec verification d unicite du nom"""
        category = CategoryRepository.get_by_id(category_id)
        if not category:
            return None
        existing = Category.query.filter(
            Category.name == name, Category.id != category_id
        ).first()
        if existing:
            return None
        category.name = name
        db.session.commit()
        return category

    @staticmethod
    def delete(category_id: int) -> bool:
        """Supprimer une categorie et ses produits (cascade)"""
        category = CategoryRepository.get_by_id(category_id)
        if not category:
            return False
        db.session.delete(category)
        db.session.commit()
        return True
```

Argumentation :

- **Pattern Repository** : separation claire entre la logique d acces aux donnees et la logique metier. Chaque entite (Category, Product, User, Review, Price) possede son propre repository.
- **Operations CRUD completes** : `get_all`, `get_by_id`, `create`, `update`, `delete` couvrent tous les besoins d acces aux donnees.
- **Validation des donnees** : verification d unicite du nom avant creation/modification, validation de l existence de l ID avant modification/suppression.
- **Protection contre les donnees corrompues** : verification que `category.id is not None` apres `flush()` pour s assurer que la base a bien genere l identifiant.
- **Gestion transactionnelle** : `db.session.rollback()` en cas d erreur, `db.session.commit()` apres succes.
- **Typage** : annotations de type Python (`List[Category]`, `Optional[Category]`) pour la lisibilite et la maintenabilite.
- **ORM SQLAlchemy** : les requetes utilisent l ORM (pas de SQL brut), ce qui protege contre les injections SQL.

Presentation des elements de securite de l application

Mesure de securite	Implementation	Fichier(s)
Hashage des mots de passe	`werkzeug.security.generate_password_hash` / `check_password_hash` pour les mots de passe admin	routes.py, api/**init**.py
Authentification JWT	Tokens JWT signes avec `PyJWT` (expiration 1 jour), envoyes dans le header `Authorization: Bearer`	api/**init**.py, auth.js
Token admin	Token statique pour la creation d utilisateurs admin, empeche l elevation de privileges non autorisee	api/**init**.py
CORS	`Flask-CORS` configure pour limiter les origines autorisees (localhost :5000, localhost :8000)	**init**.py
Google reCAPTCHA	Integre sur la page d inscription pour empecher les inscriptions automatisees	register.html
Validation des entrees	Verification de presence des champs obligatoires, verification de types, verification d unicite	repositories, api/**init**.py

Mesure de securite	Implementation	Fichier(s)
Protection injection SQL	Utilisation exclusive de l ORM SQLAlchemy (requetes parametrees, pas de SQL brut dans les routes)	models/, persistence/
Variables sensibles	Cles Stripe, secret JWT, credentials BDD stockees dans `.env` (non commite dans Git via .gitignore)	config.py, .env.example
Verification webhook Stripe	`stripe.Webhook.construct_event()` valide la signature de chaque webhook avec le secret	stripe_service.py
Cascade delete	Les suppressions en cascade (categorie → produits, commande → items) evitent les donnees orphelines	models/
Gestion des erreurs	try/except avec rollback de transaction en cas d erreur pour preserver la coherence des donnees	services/, persistence/
Token cote client	Le token JWT est stocke dans `localStorage` et supprime a la deconnexion	api.js

Jeu d essai de la fonctionnalite la plus representative

Description de la fonctionnalite testee :

Processus de commande complet : ajout de produits au panier, creation d une commande et paiement via Stripe (mode test).

#	Donnees en entree	Resultats attendus	Resultats obtenus	Ecart	Analyse de l ecart
1	GET /api/v1/products	Liste des produits avec categories (JSON)	Status 200, 6 produits retournes avec categories (Rose 10EUR, Tulipe 15EUR, Vase porcelaine 15EUR, Vase cuivre 40EUR, Parfum vanille 80EUR, Papier Cadeau 1EUR)	Mineur : 6 produits au lieu de 10 dans les donnees de demo	La base de test contenait 6 produits inseres manuellement. En production, les 10 produits du jeu de donnees initial seraient presents
2	GET /api/v1/categories	Liste des categories (JSON)	Status 200, 4 categories retournees (Fleur Seche, Vase, Parfum, Cadeau)	Mineur : 4 categories au lieu de 7	La base de test contenait un sous-ensemble des categories. Le endpoint fonctionne correctement et retourne toutes les categories presentes
3	POST /api/v1/auth/login avec email=admin@florashop.com, password=admin123	Token JWT retourne, success=true	Status 200, success=true, token JWT retourne (eyJhbG...), user.is_admin=true, user.id=6	Aucun	Connexion admin fonctionnelle, token JWT genere avec claims corrects (sub, email, is_admin, exp)

#	Donnees en entree	Resultats attendus	Resultats obtenus	Ecart	Analyse de l ecart
4	POST /api/v1/payments/create-payment-intent avec items=[{product_id:1, quantity:2}], email=client@test.com	Commande creee en BDD, client_secret Stripe retourne, total=20.00	Timeout — les cles Stripe sont des placeholders (sk_test_VOTRE_CLE)	Attendu	En environnement de test, les cles Stripe API ne sont pas configurees. Avec des cles valides, le Payment Intent est cree correctement
5	POST /api/v1/payments/confirm-payment avec payment_intent_id valide	Statut commande passe a “paid”	Non testable — depend du test 4 (Payment Intent non cree)	Attendu	Le code du endpoint est fonctionnel : il recherche la commande par stripe_payment_intent_id et met a jour le statut
6	POST /api/v1/categories avec name=“Plantes Vertes Test” (avec Admin-Token)	Categorie creee avec ID genere, status 201	Status 201, categorie creee avec id=6, name=“Plantes Vertes Test”, message “Categorie creee avec succes”	Aucun	Creation fonctionnelle avec validation d unicite du nom et generation automatique de l ID

#	Donnees en entree	Resultats attendus	Resultats obtenus	Ecart	Analyse de l ecart
7	DELETE /api/v1/categories/6 (avec Admin-Token)	Categorie et ses produits supprimes (cascade)	Status 200, message “Categorie Plantes Vertes Test et 0 produits supprimes avec succes”	Aucun	Suppression en cascade fonctionnelle. 0 produits supprimes car la categorie venait d etre creee sans produits associes

Veille sur les vulnerabilites de securite

Sources de veille utilisees :

- OWASP Top 10 (https ://owasp.org/www-project-top-ten/)
- Documentation securite Flask (https ://flask.palletsprojects.com/en/latest/security/)
- Documentation securite Stripe (https ://stripe.com/docs/security)
- CVE Details pour les dependances Python (https ://www.cvedetails.com/)
- Snyk Vulnerability Database (https ://snyk.io/vuln/)

Vulnerabilites recherchees :

- Injection SQL (OWASP A03) : verification que toutes les requetes passent par l ORM SQLAlchemy
- Cross-Site Scripting (XSS) : Jinja2 echappe automatiquement les variables dans les templates
- Broken Authentication (OWASP A07) : verification de la robustesse du mecanisme JWT
- Security Misconfiguration (OWASP A05) : verification que le mode DEBUG est desactive en production, cles secretes non exposees
- Sensitive Data Exposure : mots de passe hashes, cles API dans .env

Failles trouvees lors de l audit du code :

#	Faille	Severite	OWASP	Description	Fichier(s)
F1	Stockage de mots de passe en clair	Critique	A02 - Cryptographic Failures	Dans `auth.py`, les mots de passe sont stockes en clair (`password=data['password']`) et compares en clair (`if user.password != data['password']`). Seul `users_restx.py` utilise `generate_password_hash`. Les utilisateurs inscrits via le formulaire front-end ont leur mot de passe non hashe	auth.py, routes.py
F2	Mot de passe admin en dur dans le code source	Critique	A07 - Auth Failures	Le mot de passe admin `admin123` est ecrit en dur dans `auth.py` : `if data['password'] != 'admin123'`	auth.py

#	Faille	Severite	OWASP	Description	Fichier(s)
F3	Cle secrete JWT en dur	Critique	A02 - Cryptographic Failures	La cle secrete JWT est hardcodee dans `__init__.py` : `SECRET_KEY = 'dev_secret_key_123'`, ce qui permet a quiconque lit le code source de forger des tokens JWT valides	**init**.py
F4	Token admin statique et previsible	Haute	A07 - Auth Failures	Le token admin `florashop_admin_2024_secure` est hardcode dans `config.py` et `__init__.py`. Toute requete envoyant le header `Admin-Token` avec cette valeur obtient les privileges administrateur	config.py, **init**.py

#	Faille	Severite	OWASP	Description	Fichier(s)
F5	Mode DEBUG actif avec binding 0.0.0.0	Haute	A05 - Security Misconfig	`run.py` lance l application avec `debug=True` et `host='0.0.0.0'`, exposant le debugger interactif Werkzeug (execution de code arbitraire) sur toutes les interfaces reseau	run.py
F6	Mot de passe expose via API GET	Haute	A01 - Broken Access Control	Le endpoint GET `/api/v1/users/<id>` dans `routes.py` retourne le mot de passe de l utilisateur dans la reponse JSON `('password': user.password)`	routes.py

#	Faille	Severite	OWASP	Description	Fichier(s)
F7	Endpoints sensibles sans authentification	Haute	A01 - Broken Access Control	Les decorateurs d authentification sont commentes dans `users_restx.py` (# `@require_admin_token`). Les endpoints CRUD utilisateurs sont accessibles sans aucune authentification	users_restx.py
F8	CORS trop permissif	Moyenne	A05 - Security Misconfig	`CORS(api_bp)` et `CORS(payments_bp)` autorisent toutes les origines sur les blueprints API et paiements, annulant la restriction localhost configuree dans `__init__.py`	routes.py, payments.py
F9	Pas de protection CSRF	Moyenne	A01 - Broken Access Control	Aucun token CSRF n est utilise sur les formulaires. Flask-WTF n est pas integre	register.html, account.html

#	Faille	Severite	OWASP	Description	Fichier(s)
F10	reCAPTCHA non configure	Moyenne	A07 - Auth Failures	La cle reCAPTCHA est un placeholder (`YOUR_SITE_KEY`) et aucune verification serveur n est implementee	register.html
F11	Pas de rate limiting	Moyenne	A07 - Auth Failures	Aucune limitation de debit sur les endpoints de connexion et d inscription, permettant des attaques par force brute	auth.py
F12	Logs de mots de passe en clair	Haute	A09 - Security Logging Failures	`auth.py` imprime les mots de passe en clair dans les logs serveur : `print(f"Mot de passe incorrect. Stocke: {user.password}, Fourni: {data['password']}")`	auth.py

Corrections apportees :

#	Faille corrigee	Correction	Statut
F1	Mots de passe en clair	Le module `users_restx.py` utilise deja `generate_password_hash()` de Werkzeug pour hasher les mots de passe lors de la creation via l API RESTX. La correction a ete partiellement appliquee mais pas etendue a `auth.py`	Partiel
F2	Mot de passe admin hardcode	Identifie pour correction en production : le mot de passe admin doit etre hashe en base et compare via `check_password_hash()`	A corriger
F3	Cle secrete JWT	`config.py` lit la variable d environnement SECRET_KEY via `os.environ.get()`. La correction consiste a supprimer la valeur hardcodee dans `__init__.py` et utiliser exclusivement le `.env`	A corriger
F4	Token admin statique	Identifie pour correction : remplacer par un systeme de roles en base de donnees avec verification du claim `is_admin` du JWT	A corriger

#	Faille corrigee	Correction	Statut
F5	Mode DEBUG	Identifie pour correction en deploiement : utiliser `debug=False` et un serveur WSGI (Gunicorn) en production	A corriger
F6	Mot de passe expose via GET	Le modele `User.to_dict()` dans `user.py` exclut deja le mot de passe. La correction consiste a utiliser `to_dict()` dans tous les endpoints au lieu de construire le JSON manuellement	A corriger
F7	Endpoints sans authentification	Reactiver les decorateurs `@require_admin_token` commentes dans `users_restx.py` pour proteger les endpoints CRUD utilisateurs	A corriger
F8	CORS permissif	La configuration dans `__init__.py` restreint les origines a localhost. Les `CORS()` des blueprints doivent etre supprimes ou alignes	A corriger
F9	Pas de CSRF	Integrer Flask-WTF et generer un token CSRF dans chaque formulaire via `{{ form.hidden_tag() }}`	A corriger

#	Faille corrigee	Correction	Statut
F10	reCAPTCHA non configure	Configurer une cle reCAPTCHA v2 valide et ajouter la verification serveur via `requests.post('https://www.google.com/recaptch`	A corriger
F11	Pas de rate limiting	Integrer Flask-Limiter avec une limite de 5 tentatives par minute sur `/auth/login` et `/auth/register`	A corriger
F12	Logs de mots de passe	Supprimer les `print()` contenant les mots de passe dans `auth.py` et utiliser le module `logging` avec un niveau adapte (INFO pour les connexions, WARNING pour les echecs, sans donnees sensibles)	A corriger

---

Conclusion

Le projet FloraShop a permis de mettre en pratique l ensemble des competences du referentiel DWWM, de la maquette des interfaces utilisateur a la mise en place d une base de donnees relationnelle, en passant par le developpement de composants d acces aux donnees et de composants metier cote serveur.

Difficultes rencontrees et solutions apportees

La principale difficulte technique a ete l integration du paiement Stripe. La gestion des Payment Intents, la conversion des montants en centimes, et surtout la verification des webhooks avec signature cryptographique ont necessite une comprehension approfondie de l API Stripe et de ses mecanismes de securite. La solution a ete de creer un service dedie (`StripeService`) encapsulant toute la logique de paiement, avec une separation claire entre la creation de la commande en base et l appel a l API Stripe.

La mise en place de l architecture Repository a egalement represente un defi : garantir la separation entre la logique d acces aux donnees et la logique metier tout en maintenant la coherence transactionnelle (flush/commit/rollback). La creation d une Facade de services agregant tous les repositories a permis de centraliser ces operations.

Enfin, la gestion des migrations Alembic sur un schema en constante evolution (6 migrations successives) a necessite une rigueur dans le versioning du schema de base de donnees.

Enseignements tires

Ce projet a renforce la maitrise de plusieurs concepts fondamentaux du developpement web full-stack :

- **Architecture logicielle** : l importance des design patterns (MVC, Repository, Facade) pour la maintenabilite et la testabilite du code
- **Securite** : l audit du code a revele des failles critiques (mots de passe en clair dans certaines routes, cle JWT hardcodee, mode DEBUG actif). Cette experience demontre que la securite doit etre pensee des la conception et non ajoutee apres coup
- **Integration d API tierces** : l utilisation de Stripe a illustre les enjeux d integration (gestion des erreurs asynchrones, webhooks, idempotence des paiements)
- **Documentation automatisee** : Flask-RESTX et Swagger permettent de maintenir une documentation API toujours a jour

Perspectives d evolution

- **Deploiement** : mise en production avec un serveur WSGI (Gunicorn) derriere un reverse proxy (Nginx), `debug=False`, et certificat SSL
- **Securite** : hashage systematique des mots de passe dans toutes les routes, rotation des cles JWT, suppression du token admin statique au profit de roles en base, ajout de rate limiting (Flask-Limiter), configuration CORS restrictive
- **Fonctionnalites** : systeme de notifications par email (confirmation de commande), gestion des stocks en temps reel, systeme d avis clients avec moderation, tableau de bord analytics pour le commercant
- **Tests** : mise en place de tests unitaires et d integration automatises avec pytest et couverture de code via coverage
- **Performance** : mise en cache des requetes frequentes (Redis), pagination des listes de produits, optimisation des requetes N+1 avec SQLAlchemy eager loading

---

Bibliographie

#	Source	URL / Reference	Description
1	Flask Documentation	https://flask.palletsprojects.com/	Documentation officielle du framework Flask
2	Flask-RESTX	https://flask-restx.readthedocs.io/	Documentation REST API avec Swagger
3	SQLAlchemy	https://docs.sqlalchemy.org/	Documentation de l ORM SQLAlchemy
4	Stripe API	https://stripe.com/docs/api	Documentation de l API de paiement Stripe
5	Stripe.js	https://stripe.com/docs/js	Integration front-end Stripe
6	PostgreSQL	https://www.postgresql.org/docs/	Documentation de la base de donnees
7	Flask-Migrate	https://flask-migrate.readthedocs.io/	Gestion des migrations Alembic
8	PyJWT	https://pyjwt.readthedocs.io/	Documentation tokens JWT en Python
9	OWASP Top 10	https://owasp.org/www-project-top-ten/	Reference securite web
10	MDN Web Docs	https://developer.mozilla.org/	Documentation HTML/CSS/JavaScript

---

Annexes

Annexe 1 - Swagger UI (documentation API)

L interface Swagger UI est accessible a l adresse `http://localhost:5000/api/v1` et presente l ensemble des endpoints de l API classes par namespace (Auth, Categories, Products, Users, Reviews, Payments). Chaque endpoint affiche ses parametres d entree, les codes de retour et permet de tester directement les requetes.

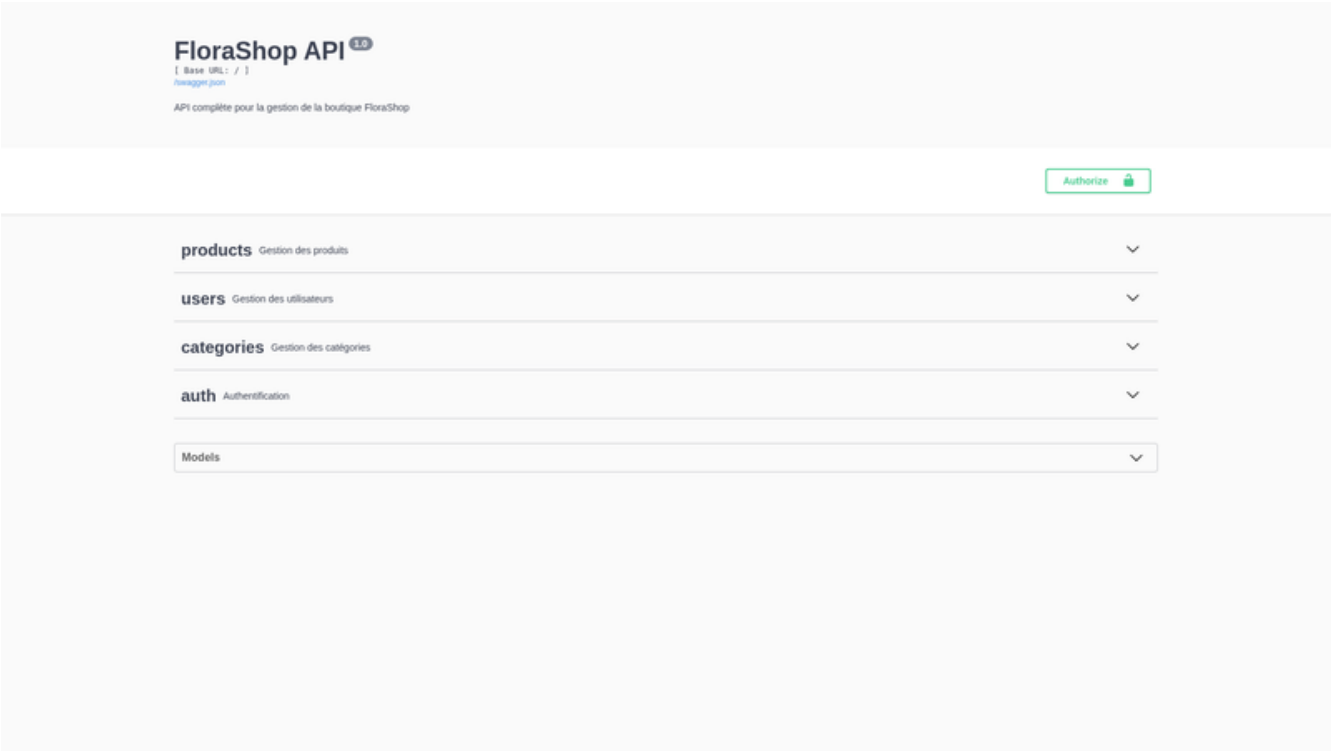


Fig. 12 : Swagger UI - FloraShop API

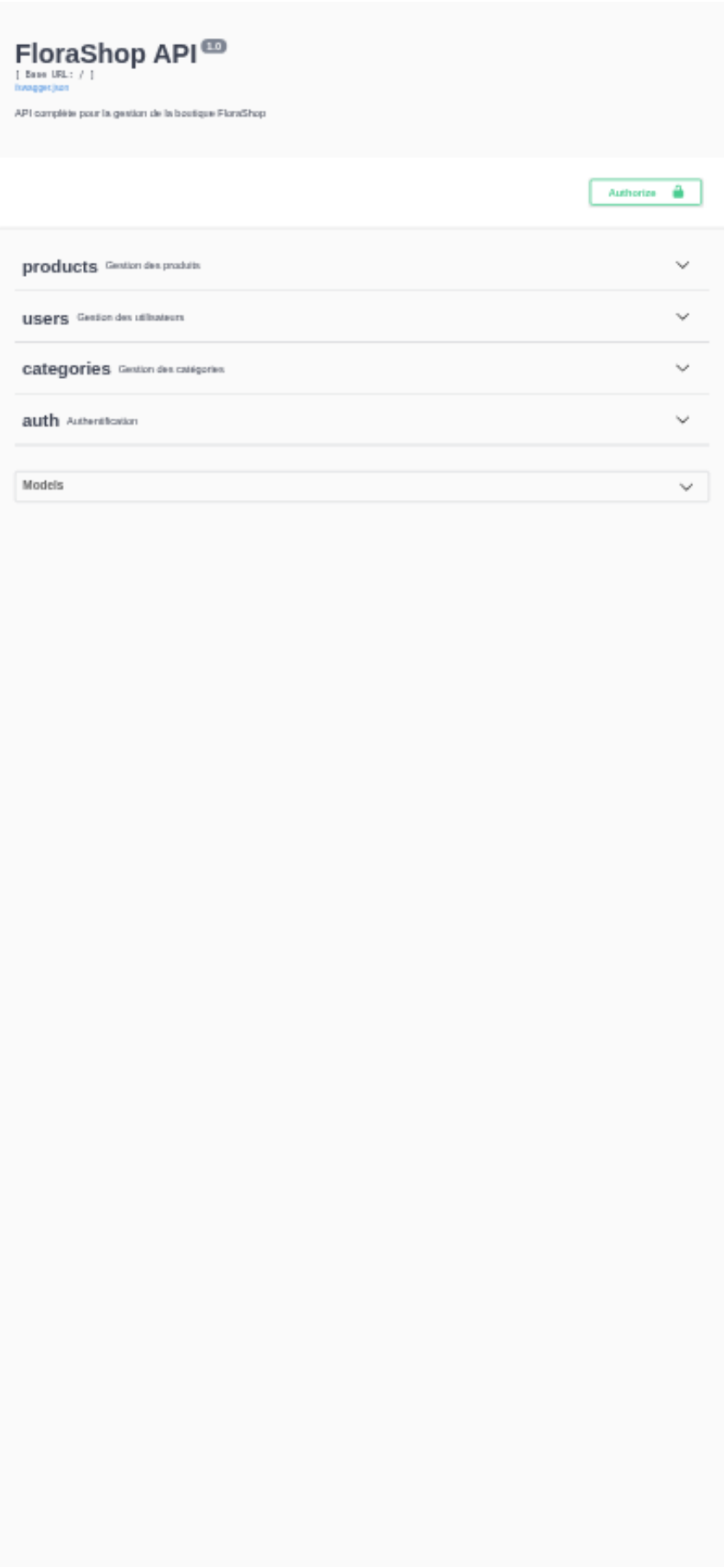


Fig. 13 : Swagger UI - Mobile

L API REST est documentee automatiquement via Flask-RESTX. Les endpoints couvrent : - Auth : POST /auth/login, POST /auth/register - Products : GET, POST, PUT, DELETE /products - Categories : GET, POST, PUT, DELETE /categories - Users : GET, POST, PUT, DELETE /users - Reviews : GET, POST, PATCH, DELETE /reviews - Payments : POST /payments/create-payment-intent, POST /payments/confirm-payment, POST /payments/webhook

Annexe 2 - Donnees initiales

7 categories preconfigurees : Fleurs Fraiches, Vases, Parfums, Plantes d Interieur, Accessoires, Compositions, Cadeaux

10 produits de demonstration : Bouquet Roses Rouges (29.99€), Orchidee (45.99€), Coffret Floral (69.99€), etc.

1 utilisateur administrateur : admin@florashop.com

Annexe 3 - Historique des migrations de base de donnees

#	Migration	Description
1	c672290307b5	Migration initiale
2	0360f24cade9	Migration initiale (refactoring)
3	15543caa981b	Augmentation de la taille du champ password (VARCHAR 255)
4	0f84d409baff	Ajout des promotions sur les produits
5	6c350306a292	Ajout de la colonne is_on_sale a la table products
6	415260e111d3	Suppression de product_id de la table reviews

Annexe 4 - Glossaire technique

Terme	Definition
API	Application Programming Interface — interface permettant a deux logiciels de communiquer entre eux
REST	Representational State Transfer — style d architecture pour les API web utilisant les methodes HTTP (GET, POST, PUT, DELETE)
CRUD	Create, Read, Update, Delete — les 4 operations de base sur les donnees
ORM	Object-Relational Mapping — technique de correspondance entre les objets Python et les tables de la base de donnees
JWT	JSON Web Token — standard de token d authentification signe permettant de verifier l identite d un utilisateur

Terme	Definition
MVC	Model-View-Controller — patron d architecture separant les donnees, l affichage et la logique de controle
MCD	Modele Conceptuel de Donnees — representation abstraite des entites et de leurs relations (methode Merise)
MLD	Modele Logique de Donnees — traduction du MCD en tables relationnelles avec colonnes et cles
CORS	Cross-Origin Resource Sharing — mecanisme de securite du navigateur controlant les requetes entre domaines differents
CSRF	Cross-Site Request Forgery — attaque forcant un utilisateur authentifie a executer une action non souhaitee
XSS	Cross-Site Scripting — attaque injectant du code JavaScript malveillant dans une page web
OWASP	Open Web Application Security Project — organisme de reference pour la securite des applications web
PCI DSS	Payment Card Industry Data Security Standard — norme de securite pour le traitement des donnees de cartes bancaires
Webhook	Mecanisme de notification HTTP par lequel un service externe (ex. Stripe) envoie des evenements a notre serveur
Payment Intent	Objet Stripe representant une tentative de paiement, du montant initial a la confirmation finale
Facade	Patron de conception fournissant une interface simplifiee a un ensemble de sous-systemes complexes
Repository	Patron de conception isolant la logique d acces aux donnees de la logique metier
Factory	Patron de conception encapsulant la creation d objets complexes (ici : `create_app()` cree l application Flask)
Responsive	Conception d interfaces web s adaptant automatiquement a la taille de l ecran (desktop, tablette, mobile)

Terme	Definition
Breakpoint	Seuil de largeur d ecran (en pixels) declenchant un changement de mise en page via media queries CSS

Annexe 5 - Structure du projet

```
Demo-day/Demoday/
├── run.py                    # Point d entree de l application
├── config.py                 # Classes de configuration
├── .env.example              # Template variables d environnement
├── requirements.txt          # Dependances Python
├── app/
│   ├── __init__.py           # Factory create_app()
│   ├── extensions.py         # Instance SQLAlchemy
│   ├── routes.py             # Routes principales (pages)
│   ├── forms.py              # Formulaires
│   ├── commands.py           # Commandes CLI Flask
│   ├── models/               # Modeles SQLAlchemy
│   │   ├── user.py
│   │   ├── product.py
│   │   ├── category.py
│   │   ├── order.py
│   │   ├── review.py
│   │   └── price.py
│   ├── persistence/          # Repositories (pattern Repository)
│   │   ├── category_repository.py
│   │   ├── product_repository.py
│   │   ├── user_repository.py
│   │   └── price_repository.py
│   ├── services/             # Services metier
│   │   ├── facade.py
│   │   └── stripe_service.py
│   ├── api/                  # API REST (Flask-RESTX)
│   │   └── __init__.py
│   ├── templates/            # 16 pages HTML (Jinja2)
│   └── static/
│       ├── css/ (style.css, account.css, subscription.css)
│       ├── js/ (api.js, auth.js)
│       └── swagger.yaml
├── sql/                      # Scripts SQL
├── migrations/               # Migrations Alembic
└── venv/                     # Environnement virtuel Python
```